4
RAG Feature Pipeline

Retrieval-augmented generation (**RAG**) is fundamental in most generative AI applications. RAG's core responsibility is to inject custom data into the **large language model** (**LLM**) to perform a given action (e.g., summarize, reformulate, and extract the injected data). You often want to use the LLM on data it wasn't trained on (e.g., private or new data). As fine-tuning an LLM is a highly costly operation, RAG is a compelling strategy that bypasses the need for constant fine-tuning to access that new data.

We will start this chapter with a theoretical part that focuses on the fundamentals of RAG and how it works. We will then walk you through all the components of a naïve RAG system: chunking, embedding, and vector DBs. Ultimately, we will present various optimizations used for an advanced RAG system. Then, we will continue exploring LLM Twin's RAG feature pipeline architecture. At this step, we will apply all the theoretical aspects we discussed at the beginning of the chapter. Finally, we will go through a practical example by implementing the LLM Twin's RAG feature pipeline based on the system design described throughout the book.

The main sections of this chapter are:

- Understanding RAG
- An overview of advanced RAG
- Exploring the LLM Twin's RAG feature pipeline architecture
- Implementing the LLM Twin's RAG feature pipeline

By the end of this chapter, you will have a clear and comprehensive understanding of what RAG is and how it is applied to our LLM Twin use case.

Understanding RAG

RAG enhances the accuracy and reliability of generative AI models with information fetched from external sources. It is a technique complementary to the internal knowledge of the LLMs. Before going into the details, let's understand what RAG stands for:

- **Retrieval**: Search for relevant data
- **Augmented**: Add the data as context to the prompt
- **Generation**: Use the augmented prompt with an LLM for generation

Any LLM is bound to understand the data it was trained on, sometimes called parameterized knowledge. Thus, even if the LLM can perfectly answer what happened in the past, it won't have access to the newest data or any other external sources on which it wasn't trained.

Let's take the most powerful model from OpenAI as an example, which, in the summer of 2024, is GPT-4o. The model is trained on data up to October 2023. Thus, if we ask what happened during the 2020 pandemic, it can be answered perfectly due to its parametrized knowledge.

However, it will not know the answer if we ask about the 2024 European Football Championship results due to its bounded parametrized knowledge. Another scenario is that it will start confidently hallucinating and provide a faulty answer.

RAG overcomes these two limitations of LLMs. It provides access to external or latest data and prevents hallucinations, enhancing generative AI models' accuracy and reliability.

Why use RAG?

We briefly explained the importance of using RAG in generative AI applications earlier. Now, we will dig deeper into the "why," following which we will focus on what a naïve RAG framework looks like.

For now, to get an intuition about RAG, you have to know that when using RAG, we inject the necessary information into the prompt to answer the initial user question. After that, we pass the augmented prompt to the LLM for the final answer. Now, the LLM will use the additional context to answer the user question.

There are two fundamental problems that RAG solves:

- Hallucinations
- Old or private information

Hallucinations

If a chatbot without RAG is asked a question about something it wasn't trained on, there is a high chance that it will give you a confident answer about something that isn't true. Let's take the 2024 European Football Championship as an example. If the model is trained up to October 2023 and we ask it something about the tournament, it will most likely come up with a random answer that is hard to differentiate between reality and truth. Even if the LLM doesn't hallucinate all the time, it raises concerns about the trustworthiness of its answers. Thus, we must ask ourselves: "When can we trust the LLM's answers?" and "How can we evaluate if the answers are correct?".

By introducing RAG, we enforce the LLM to always answer solely based on the introduced context. The LLM will act as the reasoning engine, while the additional information added through RAG will act as the single source of truth for the generated answer. By doing so, we can quickly evaluate if the LLM's answer is based on the external data or not.

Old information

Any LLM is trained or fine-tuned on a subset of the total world knowledge dataset. This is due to three main issues:

- **Private data**: You cannot train your model on data you don't own or have the right to use.
- **New data**: New data is generated every second. Thus, you would have to constantly train your LLM to keep up.
- **Costs**: Training or fine-tuning an LLM is an extremely costly operation. Hence, it is not feasible to do it on an hourly or daily basis.

RAG solves these issues, as you no longer have to constantly fine-tune your LLM on new data (or even private data). Directly injecting the necessary data to respond to user questions into the prompts that are fed to the LLM is enough to generate correct and valuable answers.

To conclude, RAG is key for a robust and flexible generative AI system. But how do we inject the right data into the prompt based on the user's questions? We will dig into the technical aspects of RAG in the next sections.

The vanilla RAG framework

Every RAG system is similar at its roots. We will first focus on understanding RAG in its simplest form. Later, we will gradually introduce more advanced RAG techniques to improve the system's accuracy. Note that we will use vanilla and naive RAG interchangeably to avoid repetition.

A RAG system is composed of three main modules independent of each other:

- **Ingestion pipeline**: A batch or streaming pipeline used to populate the vector DB
- **Retrieval pipeline**: A module that queries the vector DB and retrieves relevant entries to the user's input
- **Generation pipeline**: The layer that uses the retrieved data to augment the prompt and an LLM to generate answers

As these three components are classes or services of their own, we will dig into each separately. But for now, let's try to answer the question "How are these three modules connected?". Here is a very simplistic overview:

1. On the backend side, the ingestion pipeline runs either on a schedule or constantly to populate the vector DB with external data.
2. On the client side, the user asks a question.
3. The question is passed to the retrieval module, which preprocesses the user's input and queries the vector DB.
4. The generation pipelines use a prompt template, user input, and retrieved context to create the prompt.
5. The prompt is passed to an LLM to generate the answer.
6. The answer is shown to the user.

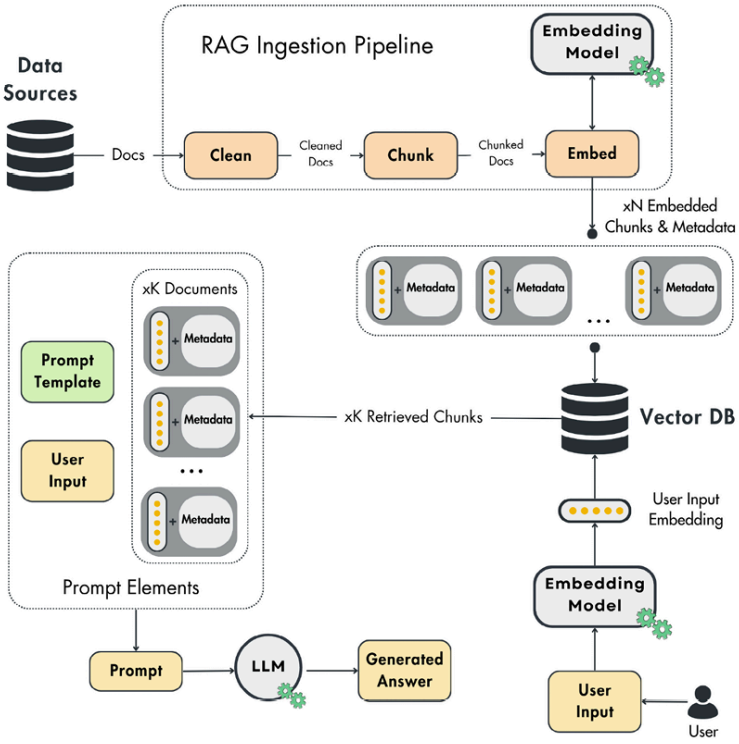

Figure 4.1: Vanilla RAG architecture

You must implement RAG in your generative AI application when you need access to any type of external information. For example, when implementing a financial assistant, you most likely need access to the latest news, reports, and prices before providing valuable answers. Or, if you build a traveling recommender, you must retrieve and parse a list of potential attractions, restaurants, and activities. At training time, LLMs don't have access to your specific data, so you will often have to implement a RAG strategy in your generative AI project. Now, let's dig into the ingestion, retrieval, and generation pipelines.

Ingestion pipeline

The RAG ingestion pipeline extracts raw documents from various data sources (e.g., data warehouse, data lake, web pages, etc.). Then, it cleans, chunks (splits into smaller sections), and embeds the documents. Ultimately, it loads the embedded chunks into a vector DB (or other similar vector storage).

Thus, the RAG ingestion pipeline is split into the following:

- The **data extraction module** gathers all the necessary data from various sources such as DBs, APIs, or web pages. This module is highly dependent on your data. It can be as easy as querying your data warehouse or something more complex such as crawling Wikipedia.
- A **cleaning layer** standardizes and removes unwanted characters from the extracted data. For example, you must remove all invalid characters from your input text, such as non-ASCII and bold and italic characters. Another popular cleaning strategy is to replace URLs with placeholders. However, your cleaning strategy will vary depending on your data source and embedding model.
- The **chunking module** splits the cleaned documents into smaller ones. As we want to pass the document's content to an embedding model, this is necessary to ensure it doesn't exceed the model's input maxi-

mum size. Also, chunking is required to separate specific regions that are semantically related. For example, when chunking a book's chapter, the most optimal way is to group similar paragraphs into the same section or chunk. By doing so, at the retrieval time, you will add only the essential data to the prompt.

- The **embedding component** uses an embedding model to take the chunk's content (text, images, audio, etc.) and project it into a dense vector packed with semantic value—more on embeddings in the *What are embeddings?* section below.
- The **loading module** takes the embedded chunks along with a metadata document. The metadata will contain essential information such as the embedded content, the URL to the source of the chunk, and when the content was published on the web. The embedding is used as an index to query similar chunks, while the metadata is used to access the information added to augment the prompt.

At this point, we have a RAG ingestion pipeline that takes raw documents as input, processes them, and populates a vector DB. The next step is to retrieve relevant data from the vector store correctly.

Retrieval pipeline

The retrieval components take the user's input (text, image, audio, etc.), embed it, and query the vector DB for similar vectors to the user's input.

The primary function of the retrieval step is to project the user's input into the same vector space as the embeddings used as an index in the vector DB. This allows us to find the top K's most similar entries by comparing the embeddings from the vector storage with the user's input vector. These entries then serve as content to augment the prompt that is passed to the LLM to generate the answer.

You must use a distance metric to compare two vectors, such as the Euclidean or Manhattan distance. But the most popular one is the cosine distance, which is equal to 1 minus the cosine of the angle between two vectors, as follows:

$$Cosine\ Distance = 1 - \cos(\theta) = 1 - \frac{A \cdot \mathrm{B}}{||\,A\,|| \cdot ||\,\mathrm{B}\,||}$$

It ranges from `-1` to `1`, with a value of `-1` when vectors **A** and **B** are in opposite directions, `0` if they are orthogonal, and `1` if they point in the same direction.

Most of the time, the cosine distance works well in non-linear complex vector spaces. However, it is essential to notice that choosing the proper distance between two vectors depends on your data and the embedding model you use.

One critical factor to highlight is that the user's input and embeddings must be in the same vector space. Otherwise, you cannot compute the distance between them. To do so, it is essential to preprocess the user input in the same way you processed the raw documents in the RAG ingestion pipeline. This means you must clean, chunk (if necessary), and embed the user's input using the same functions, models, and hyperparameters. This is similar to how you have to preprocess the data into features in the same way between training and inference; otherwise, the inference will

yield inaccurate results—a phenomenon also known as the training-serving skew.

Generation pipeline

The last step of the RAG system is to take the user's input, retrieve data, pass it to an LLM, and generate a valuable answer.

The final prompt results from a system and prompt template populated with the user's query and retrieved context. You might have a single prompt template or multiple prompt templates, depending on your application. Usually, all the prompt engineering is done at the prompt template level.

Below, you can see a dummy example of what a generic system and prompt template look like and how they are used together with the retrieval logic and the LLM to generate the final answer:

```
system_template = """
You are a helpful assistant who answers all the user's questions politely.
"""
prompt_template = """
Answer the user's question using only the provided context. If you cannot answer us
Context: {context}
User question: {user_question}
"""
user_question = "<your_question>"
retrieved_context = retrieve(user_question)
prompt = f"{system_template}\n"
prompt += prompt_template.format(context=retrieved_context, user_question=user_ques
answer  = llm(prompt)
```

As the prompt templates evolve, each change should be tracked and versioned using **machine learning operations** (**MLOps**) best practices. Thus, during training or inference time, you always know that a given answer was generated by a specific version of the LLM and prompt template(s). You can do this through Git, store the prompt templates in a DB, or use specific prompt management tools such as LangFuse.

As we've seen in the retrieval pipeline, some critical aspects that directly impact the accuracy of your RAG system are the embeddings of the external data, usually stored in vector DBs, the embedding of the user's query, and how we can find similarities between the two using functions such as the cosine distance. To better understand this part of the RAG algorithm, let's zoom in on what embeddings are and how they are computed.

What are embeddings?

Imagine you're trying to teach a computer to understand the world. Embeddings are like a particular translator that turns these things into a numerical code. This code isn't random, though, because similar words or items end up with codes that are close to each other. It's like a map where words with similar meanings are clustered together.

With that in mind, a more theoretical definition is that embeddings are dense numerical representations of objects encoded as vectors in a continuous vector space, such as words, images, or items in a recommendation system. This transformation helps capture the semantic meaning and

relationships between the objects. For instance, in **natural language processing (NLP)**, embeddings translate words into vectors where semantically similar words are positioned closely together in the vector space.

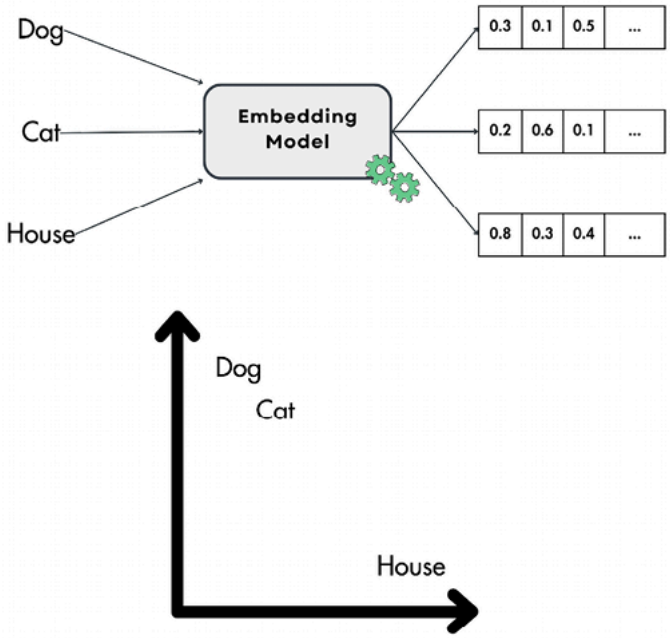

Figure 4.2: What are embeddings?

A popular method is visualizing the embeddings to understand and evaluate their geometrical relationship. As the embeddings often have more than 2 or 3 dimensions, usually between 64 and 2048, you must project them again to 2D or 3D.

For example, you can use UMAP (https://umap-learn.readthedocs.io/en/latest/index.html), a dimensionality reduction method well known for keeping the geometrical properties between the points when projecting the embeddings to 2D or 3D. Another popular algorithm for dimensionality reduction when visualizing vectors is t-SNE (https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html). However, compared to UMAP, it is more stochastic and doesn't preserve the topological relationships between the points.

A dimensionality reduction algorithm, such as PCA, UMAP, and t-SNE, is a mathematical technique used to reduce the number of input variables or features in a dataset while preserving the data's essential patterns, structure, and relationships. The goal is to transform high-dimensional data into a lower-dimensional form, making it easier to visualize, interpret, and process while minimizing the loss of important information. These methods help to address the "curse of dimensionality," improve computational efficiency, and often enhance the performance of ML algorithms.

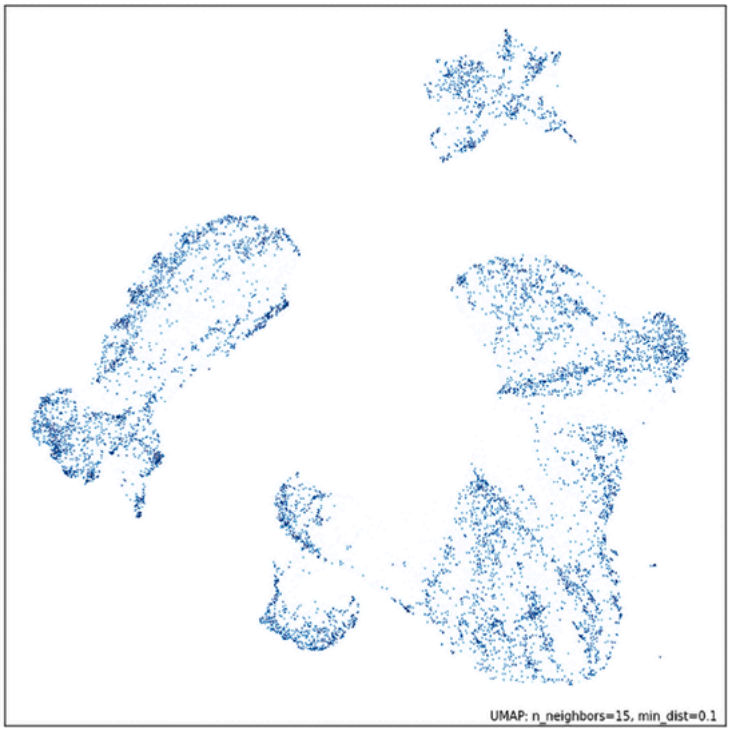

Figure 4.3: Visualize embeddings using UMAP (Source: UMAP's documentation)

Why embeddings are so powerful

Firstly, ML models work only with numerical values. This is not a problem when working with tabular data, as the data is often in numerical form or can easily be processed into numbers. Embeddings come in handy when we want to feed words, images, or audio data into models.

For instance, when working with transformer models, you tokenize all your text input, where each token has an embedding associated with it. The beauty of this process lies in its simplicity; the input to the transformer is a sequence of embeddings, which can be easily and confidently interpreted by the dense layers of the neural network.

Based on this example, you can use embeddings to encode any categorical variable and feed it to an ML model. But why not use other simple methods, such as **one-hot encoding?** When working with categorical variables with high cardinality, such as language vocabularies, you will suffer from the curse of dimensionality when using other classical methods. For example, if your vocabulary has 10,000 tokens, then only one token will have a length of 10,000 after applying one-hot encoding. If the input sequence has N tokens, that will become N * 10,000 input parameters. If N >= 100, often, when inputting text, the input is too large to be usable. Another issue with other classical methods that don't suffer from the curse of dimensionality, such as **hashing**, is that you lose the semantic relationships between the vectors.

> **One-hot encoding** is a technique that converts categorical variables into a binary matrix representation. Each category is represented as a unique binary vector. For each categorical variable, a binary vector is created with a length equal to the number of unique categories, where all values are zero except for the index corresponding to the specific category,

which is set to one. The method preserves all information about the categories. It is simple and interpretable. However, a significant disadvantage is that it can lead to a high-dimensional feature space if the categorical variable has many unique values, making the method impractical.

Feature hashing, also known as hashing encoding or the "hash trick," is a technique used to convert categorical variables into numerical features by applying a hash function to the category values. Compared to one-hot encoding, the method is not bound to the number of unique categories, but it reduces the dimensionality of the feature space by mapping categories into a fixed number of bins or buckets. Thus, it reduces the dimensionality of the feature space, which is particularly useful when dealing with high-cardinality categorical variables. This makes it efficient in terms of memory usage and computational time. However, there is a risk of collisions, where different categories might map to the same bin, leading to a loss of information. The mapping makes the method uninterpretable. Also, it is difficult to understand the relationship between the original categories and the hashed features.

Embeddings help us encode categorical variables while controlling the output vector's dimension. They also use ingenious ways to condense information into a lower dimension space than naive hashing tricks.

Secondly, embedding your input reduces the size of its dimension and condenses all of its semantic meaning into a dense vector. This is an extremely popular technique when working with images, where a CNN encoder module maps the high-dimensional meaning into an embedding, which is later processed by a CNN decoder that performs the classification or regression steps.

The following image shows a typical CNN layout. Imagine tiny squares within each layer. Those are the "receptive fields." Each square feeds information to a single neuron in the previous layer. As you move through the network, two key things are happening:

- **Shrinking the picture**: Special "subsampling" operations make the layers smaller, focusing on essential details.
- **Learning features**: "Convolution" operations, on the other hand, actually increase the layer size as the network learns more complex features from the image.

Finally, a fully connected layer at the end takes all this processed information and transforms it into the final vector embedding, a numerical image representation.

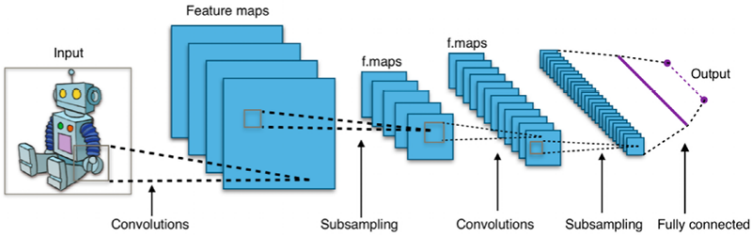

Figure 4.4: Creating embeddings from an image using a CNN (Image source)

The preceding image is sourced from *Wikimedia Commons* (https://commons.wikimedia.org/wiki/File:Typical_cnn.png) and licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0: https://creativecommons.org/licenses/by-sa/4.0/deed.en).

How are embeddings created?

Embeddings are created by deep learning models that understand the context and semantics of your input and project it into a continuous vector space.

Various deep learning models can be used to create embeddings, varying by the data input type. Thus, it is fundamental to understand your data and what you need from it before picking an embedding model.

For example, when working with text data, one of the early methods used to create embeddings for your vocabulary is Word2Vec and GloVe. These are still popular methods used today for simpler applications.

Another popular method is to use encoder-only transformers, such as BERT, or other methods from its family, such as RoBERTa. These models leverage the encoder of the transformer architecture to smartly project your input into a dense vector space that can later be used as embeddings.

To quickly compute the embeddings in Python, you can conveniently leverage the Sentence Transformers Python package (also available in Hugging Face's transformer package). This tool provides a user-friendly interface, making the embedding process straightforward and efficient.

In the code snippet below, you can see how we loaded a model from SentenceTransformer, computed the embeddings for three sentences, and, ultimately, computed the cosine similarity between them. The similarity between one sentence and itself is always 1. Also, the similarity between the first and second sentences is approximately 0, as the sentences have nothing in common. In contrast, the value between the first and third one is higher as there is some overlapping context:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
sentences = [
"The dog sits outside waiting for a treat.",
"I am going swimming.",
"The dog is swimming."
]
```

```
embeddings = model.encode(sentences)
print(embeddings.shape)
# Output: [3, 384]
similarities = model.similarity(embeddings, embeddings)
print(similarities)
# Output:
# tensor([[ 1.0000, -0.0389, 0.2692],
# [-0.0389, 1.0000, 0.3837],
# [ 0.2692, 0.3837, 1.0000]])
#
# similarities[0, 0] = The similarity between the first sentence and itself.
# similarities[0, 1] = The similarity between the first and second sentence.
# similarities[2, 1] = The similarity between the third and second sentence.
```

The source code for the preceding snippet can be found at https://github.com/PacktPublishing/LLM-Engineering/blob/main/code_snippets/08_text_embeddings.py.

The examples in the embeddings section can be run within the virtual environment used across the book, as it contains all the required dependencies.

The best-performing embedding model can change with time and your specific use case. You can find particular models on the **Massive Text Embedding Benchmark** (**MTEB**) on Hugging Face. Depending on your needs, you can consider the best-performing model, the one with the best accuracy, or the one with the smallest memory footprint. This decision is solely based on your requirements (e.g., accuracy and hardware). However, Hugging Face and SentenceTransformer make switching between different models straightforward. Thus, you can always experiment with various options.

When working with images, you can embed them using **convolutional neural networks** (**CNNs**). Popular CNN networks are based on the ResNet architecture. However, we can't directly use image embedding techniques for audio recordings. Instead, we can create a visual representation of the audio, such as a spectrogram, and then apply image embedding models to those visuals. This allows us to capture the essence of images and sounds in a way computers can understand.

By leveraging models like CLIP, you can practically embed a piece of text and an image in the same vector space. This allows you to find similar images using a sentence as input, or the other way around, demonstrating the practicality of CLIP.

In the following code snippet, we use CLIP to encode a crazy cat image and three sentences. Ultimately, we use cosine similarity to compute the resemblance between the picture and the sentences:

```
from io import BytesIO
import requests
from PIL import Image
from sentence_transformers import SentenceTransformer
response = requests.get(
"https://github.com/PacktPublishing/LLM-Engineering/blob/main/images/crazy_cat.jpg?
)
image = Image.open(BytesIO(response.content))
model = SentenceTransformer("clip-ViT-B-32")
```

```
img_emb = model.encode(image)
text_emb = model.encode(
["A crazy cat smiling.",
"A white and brown cat with a yellow bandana.",
"A man eating in the garden."]
)
print(text_emb.shape) # noqa
# Output: (3, 512)
similarity_scores = model.similarity(img_emb, text_emb)
print(similarity_scores) # noqa
# Output: tensor([[0.3068, 0.3300, 0.1719]])
```

The source code can be found at https://github.com/PacktPublishing/LLM-Engineering/blob/main/code_snippets/08_text_image_embeddings.py.

Here, we provided a small introduction to how embeddings can be computed. The realm of specific implementations is vast, but what is important to know is that embeddings can be computed for most digital data categories, such as words, sentences, documents, images, videos, and graphs.

It's crucial to grasp that you must use specialized models when you need to compute the distance between two different data categories, such as the distance between the vector of a sentence and of an image. These models are designed to project both data types into the same vector space, such as CLIP, ensuring accurate distance computation.

Applications of embeddings

Due to the generative AI revolution, which uses RAG, embeddings have become extremely popular in information retrieval tasks, such as semantic search for text, code, images, and audio, and long-term memory of agents. But before generative AI, embeddings were already heavily used in:

- Representing categorical variables (e.g., vocabulary tokens) that are fed to an ML model
- Recommender systems by encoding the users and items and finding their relationship
- Clustering and outlier detection
- Data visualization by using algorithms such as UMAP
- Classification by using the embeddings as features
- Zero-shot classification by comparing the embedding of each class and picking the most similar one

The last step to fully understanding how RAG works is to examine vector DBs and how they leverage embeddings to retrieve data.

More on vector DBs

Vector DBs are specialized DBs designed to efficiently store, index, and retrieve vector embeddings. Traditional scalar-based DBs struggle with the complexity of vector data, making vector DBs crucial for tasks like real-time semantic search.

While standalone vector indices like FAISS are effective for similarity search, they lack vector DBs' comprehensive data management capabilities. Vector DBs support CRUD operations, metadata filtering, scalability, real-time updates, backups, ecosystem integration, and robust data security, making them more suited for production environments than standalone indices.

How does a vector DB work?

Think of how you usually search a DB. You type in something specific, and the system spits out the exact match. That's how traditional DBs work. Vector DBs are different. Instead of perfect matches, we look for the closest neighbors of the query vector. Under the hood, a vector DB uses **approximate nearest neighbor** (**ANN**) algorithms to find these close neighbors.

While ANN algorithms don't return the top matches for a given search, standard nearest neighbor algorithms are too slow to work in practice. Also, it is shown empirically that using only approximations of the top matches for a given input query works well enough. Thus, the trade-off between accuracy and latency ultimately favors ANN algorithms.

This is a typical workflow of a vector DB:

1. **Indexing vectors**: Vectors are indexed using data structures optimized for high-dimensional data. Common indexing techniques include **hierarchical navigable small world** (**HNSW**), random projection, **product quantization** (**PQ**), and **locality-sensitive hashing** (**LSH**).
2. **Querying for similarity**: During a search, the DB queries the indexed vectors to find those most similar to the input vector. This process involves comparing vectors based on similarity measures such as cosine similarity, Euclidean distance, or dot product. Each has unique advantages and is suitable for different use cases.
3. **Post-processing results**: After identifying potential matches, the results undergo post-processing to refine accuracy. This step ensures that the most relevant vectors are returned to the user.

Vector DBs can filter results based on metadata before or after the vector search. Both approaches have trade-offs in terms of performance and accuracy. The query also depends on the metadata (along with the vector index), so it contains a metadata index user for filtering operations.

Algorithms for creating the vector index

Vector DBs use various algorithms to create the vector index and manage searching data efficiently:

- **Random projection**: Random projection reduces the dimensionality of vectors by projecting them into a lower-dimensional space using a random matrix. This technique preserves the relative distances between vectors, facilitating faster searches.
- **PQ**: PQ compresses vectors by dividing them into smaller sub-vectors and then quantizing these sub-vectors into representative codes. This reduces memory usage and speeds up similarity searches.

- **LSH**: LSH maps similar vectors into buckets. This method enables fast approximate nearest neighbor searches by focusing on a subset of the data, reducing the computational complexity.
- **HNSW**: HNSW constructs a multi-layer graph where each node represents a set of vectors. Similar nodes are connected, allowing the algorithm to navigate the graph and find the nearest neighbors efficiently.

These algorithms enable vector DBs to efficiently handle complex and large-scale data, making them a perfect fit for a variety of AI and ML applications.

DB operations

Vector DBs also share common characteristics with standard DBs to ensure high performance, fault tolerance, and ease of management in production environments. Key operations include:

- **Sharding and replication**: Data is partitioned (sharded) across multiple nodes to ensure scalability and high availability. Data replication across nodes helps maintain data integrity and availability in case of node failures.
- **Monitoring**: Continuous monitoring of DB performance, including query latency and resource usage (RAM, CPU, disk), helps maintain optimal operations and identify potential issues before they impact the system.
- **Access control**: Implementing robust access control mechanisms ensures that only authorized users can access and modify data. This includes role-based access controls and other security protocols to protect sensitive information.
- **Backups**: Regular DB backups are critical for disaster recovery. Automated backup processes ensure that data can be restored to a previous state in case of corruption or loss.

An overview of advanced RAG

The vanilla RAG framework we just presented doesn't address many fundamental aspects that impact the quality of the retrieval and answer generation, such as:

- Are the retrieved documents relevant to the user's question?
- Is the retrieved context enough to answer the user's question?
- Is there any redundant information that only adds noise to the augmented prompt?
- Does the latency of the retrieval step match our requirements?
- What do we do if we can't generate a valid answer using the retrieved information?

From the questions above, we can draw two conclusions. The first one is that we need a robust evaluation module for our RAG system that can quantify and measure the quality of the retrieved data and generate answers relative to the user's question. We will discuss this topic in more detail in *Chapter 9*. The second conclusion is that we must improve our RAG framework to address the retrieval limitations directly in the algorithm. These improvements are known as advanced RAG.

The vanilla RAG design can be optimized at three different stages:

- **Pre-retrieval**: This stage focuses on how to structure and preprocess your data for data indexing optimizations as well as query optimizations.
- **Retrieval**: This stage revolves around improving the embedding models and metadata filtering to improve the vector search step.
- **Post-retrieval**: This stage mainly targets different ways to filter out noise from the retrieved documents and compress the prompt before feeding it to an LLM for answer generation.

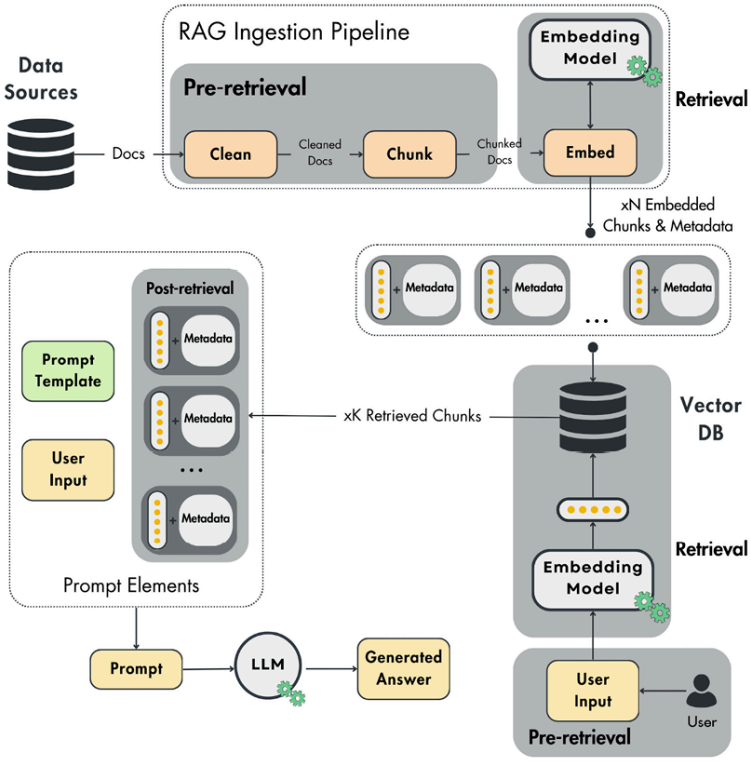

Figure 4.5: The three stages of advanced RAG

This section is not meant to be an exhaustive list of all the advanced RAG methods available. The goal is to build an intuition about what can be optimized. We will use only examples based on text data, but the principles of advanced RAG remain the same regardless of the data category. Now, let's zoom in on all three components.

Pre-retrieval

The pre-retrieval steps are performed in two different ways:

- **Data indexing**: It is part of the RAG ingestion pipeline. It is mainly implemented within the cleaning or chunking modules to preprocess the data for better indexing.
- **Query optimization**: The algorithm is performed directly on the user's query before embedding it and retrieving the chunks from the vector DB.

As we index our data using embeddings that semantically represent the content of a chunked document, most of the **data indexing** techniques focus on better preprocessing and structuring the data to improve retrieval efficiency, such as:

- **Sliding window**: The sliding window technique introduces overlap between text chunks, ensuring that important context near chunk boundaries is retained, which enhances retrieval accuracy. This is particularly beneficial in domains like legal documents, scientific papers, customer support logs, and medical records, where critical information often spans multiple sections. The embedding is computed on the chunk along with the overlapping portion. Hence, the sliding window improves the system's ability to retrieve relevant and coherent information by maintaining context across boundaries.
- **Enhancing data granularity**: This involves data cleaning techniques like removing irrelevant details, verifying factual accuracy, and updating outdated information. A clean and accurate dataset allows for sharper retrieval.
- **Metadata**: Adding metadata tags like dates, URLs, external IDs, or chapter markers helps filter results efficiently during retrieval.
- **Optimizing index structures**: It is based on different data index methods, such as various chunk sizes and multi-indexing strategies.
- **Small-to-big**: The algorithm decouples the chunks used for retrieval and the context used in the prompt for the final answer generation. The algorithm uses a small sequence of text to compute the embedding while preserving the sequence itself and a wider window around it in the metadata. Thus, using smaller chunks enhances the retrieval's accuracy, while the larger context adds more contextual information to the LLM.

The intuition behind this is that if we use the whole text for computing the embedding, we might introduce too much noise, or the text could contain multiple topics, which results in a poor overall semantic representation of the embedding.

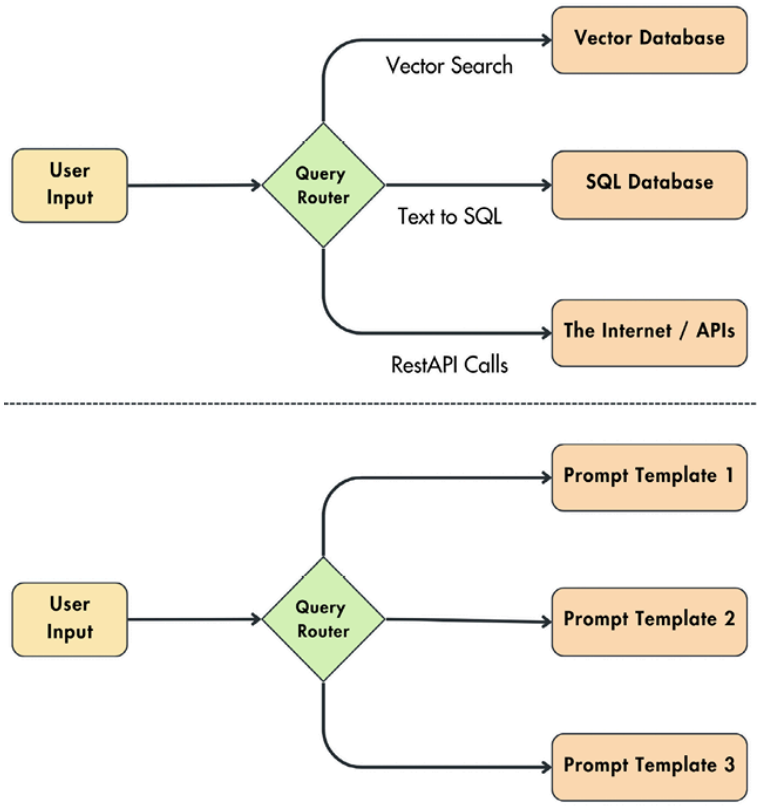

Figure 4.6: Query routing

On the **query optimization** side, we can leverage techniques such as query routing, query rewriting, and query expansion to refine the retrieved information for the LLM further:

- **Query routing**: Based on the user's input, we might have to interact with different categories of data and query each category differently. Query rooting is used to decide what action to take based on the user's input, similar to if/else statements. Still, the decisions are made solely using natural language instead of logical statements.

As illustrated in Figure 4.6, let's assume that, based on the user's input, to do RAG, we can retrieve additional context from a vector DB using vector search queries, a standard SQL DB by translating the user query to an SQL command, or the internet by leveraging REST API calls. The query router can also detect whether a context is required, helping us avoid making redundant calls to external data storage. Also, a query router can be used to pick the best prompt template for a given input. For example, in the LLM Twin use case, depending on whether the user wants an article paragraph, a post, or a code snippet, you need different prompt templates to optimize the creation process. The routing usually uses an LLM to decide what route to take or embeddings by picking the path with the most similar vectors. To summarize, query routing is identical to an if/else statement but much more versatile as it works directly with natural language.

- **Query rewriting**: Sometimes, the user's initial query might not perfectly align with the way your data is structured. Query rewriting tackles this by reformulating the question to match the indexed information better. This can involve techniques like:
  - **Paraphrasing**: Rephrasing the user's query while preserving its meaning (e.g., "What are the causes of climate change?" could be rewritten as "Factors contributing to global warming").
  - **Synonym substitution**: Replacing less common words with synonyms to broaden the search scope (e.g., " joyful" could be rewritten as "happy").
  - **Sub-queries**: For longer queries, we can break them down into multiple shorter and more focused sub-queries. This can help the retrieval stage identify relevant documents more precisely.
- **Hypothetical document embeddings** (**HyDE**): This technique involves having an LLM create a hypothetical response to the query. Then, both the original query and the LLM's response are fed into the retrieval stage.
- **Query expansion**: This approach aims to enrich the user's question by adding additional terms or concepts, resulting in different perspectives of the same initial question. For example, when searching for "disease," you can leverage synonyms and related terms associated with the original query words and also include "illnesses" or "ailments."
- **Self-query**: The core idea is to map unstructured queries into structured ones. An LLM identifies key entities, events, and relationships within the input text. These identities are used as filtering parameters to reduce the vector search space (e.g., identify cities within the query, for example, "Paris," and add it to your filter to reduce your vector search space).

Both data indexing and query optimization pre-retrieval optimization techniques depend highly on your data type, structure, and source. Thus, as with any data processing pipeline, no method always works, as every use case has its own particularities and gotchas. Optimizing your pre-retrieval RAG layer is experimental. Thus, what is essential is to try multiple methods (such as the ones enumerated in this section), reiterate, and observe what works best.

Retrieval

The retrieval step can be optimized in two fundamental ways:

- **Improving the embedding models** used in the RAG ingestion pipeline to encode the chunked documents and, at inference time, transform the user's input.
- **Leveraging the DB's filter and search features.** This step will be used solely at inference time when you have to retrieve the most similar chunks based on user input.

Both strategies are aligned with our ultimate goal: to enhance the vector search step by leveraging the semantic similarity between the query and the indexed data.

When improving the embedding models, you usually have to fine-tune the pre-trained embedding models to tailor them to specific jargon and nuances of your domain, especially for areas with evolving terminology or rare terms.

Instead of fine-tuning the embedding model, you can leverage instructor models (https://huggingface.co/hkunlp/instructor-xl) to guide the embedding generation process with an instruction/prompt aimed at your domain. Tailoring your embedding network to your data using such a model can be a good option, as fine-tuning a model consumes more computing and human resources.

In the code snippet below, you can see an example of an Instructor model that embeds article titles about AI:

```
from InstructorEmbedding import INSTRUCTOR
model = INSTRUCTOR("hkunlp/instructor-base")
sentence = "RAG Fundamentals First"
instruction = "Represent the title of an article about AI:"
embeddings = model.encode([[instruction, sentence]])
print(embeddings.shape) # noqa
# Output: (1, 768)
```

The source code can be found at https://github.com/PacktPublishing/LLM-Engineering/blob/main/code_snippets/08_instructor_embeddings.py.

To run the instructor code, you have to create a different virtual environment and activate it:

```
python3 -m venv instructor_venv && source instructor_venv/bin/activate
```

And install the required Python dependencies:

```
pip install sentence-transformers==2.2.2 InstructorEmbedding==1.0.1
```

On the other side of the spectrum, here is how you can improve your retrieval by leveraging classic filter and search DB features:

- **Hybrid search**: This is a vector and keyword-based search blend. Keyword-based search excels at identifying documents containing specific keywords. When your task demands pinpoint accuracy and the retrieved information must include exact keyword matches, hybrid search shines. Vector search, while powerful, can sometimes struggle with finding exact matches, but it excels at finding more general semantic similarities. You leverage both keyword matching and semantic similarities by combining the two methods. You have a parameter, usually called alpha, that controls the weight between the two methods. The algorithm has two independent searches, which are later normalized and unified.
- **Filtered vector search**: This type of search leverages the metadata index to filter for specific keywords within the metadata. It differs from a hybrid search in that you retrieve the data once using only the vector index and perform the filtering step before or after the vector search to reduce your search space.

In practice, on the retrieval side, you usually start with filtered vector search or hybrid search, as they are fairly quick to implement. This approach gives you the flexibility to adjust your strategy based on performance. If the results are not as expected, you can always fine-tune your embedding model.

Post-retrieval

The post-retrieval optimizations are solely performed on the retrieved data to ensure that the LLM's performance is not compromised by issues such as limited context windows or noisy data. This is because the retrieved context can sometimes be too large or contain irrelevant information, both of which can distract the LLM.

Two popular methods performed at the post-retrieval step are:

- **Prompt compression**: Eliminate unnecessary details while keeping the essence of the data.
- **Re-ranking**: Use a cross-encoder ML model to give a matching score between the user's input and every retrieved chunk. The retrieved items are sorted based on this score. Only the top N results are kept as the most relevant. As you can see in *Figure 4.7*, this works because the re-ranking model can find more complex relationships between the user input and some content than a simple similarity search. However, we can't apply this model at the initial retrieval step because it is costly. That is why a popular strategy is to retrieve the data using a similarity distance between the embeddings and refine the retrieved information using a re-raking model, as illustrated in Figure 4.8.

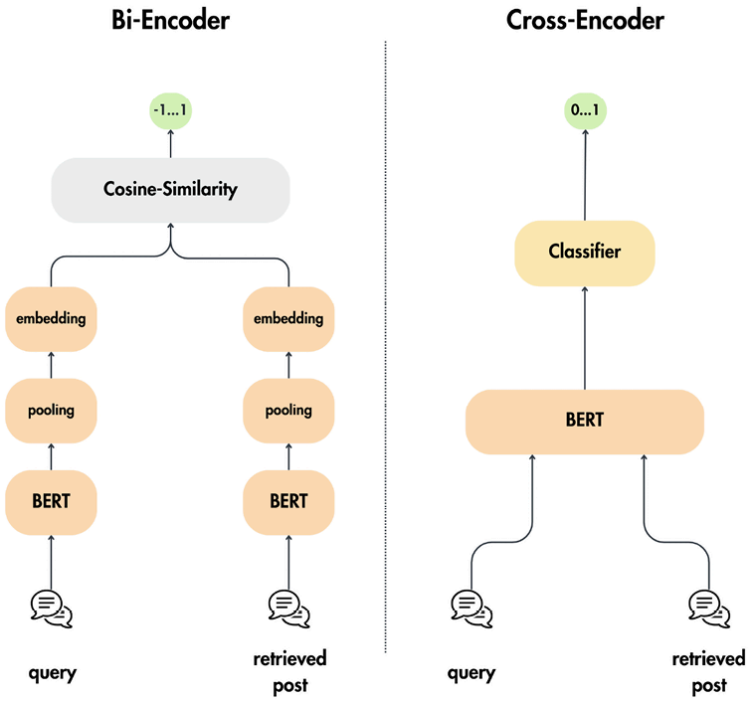

Figure 4.7: Bi-encoder (the standard embedding model) versus cross-encoder

The abovementioned techniques are far from an exhaustive list of all potential solutions. We used them as examples to get an intuition on what you can (and should) optimize at each step in your RAG workflow. The truth is that these techniques can vary tremendously by the type of data you work with.

For example, if you work with multi-modal data such as text and images, most of the techniques from earlier won't work as they are designed for text only.

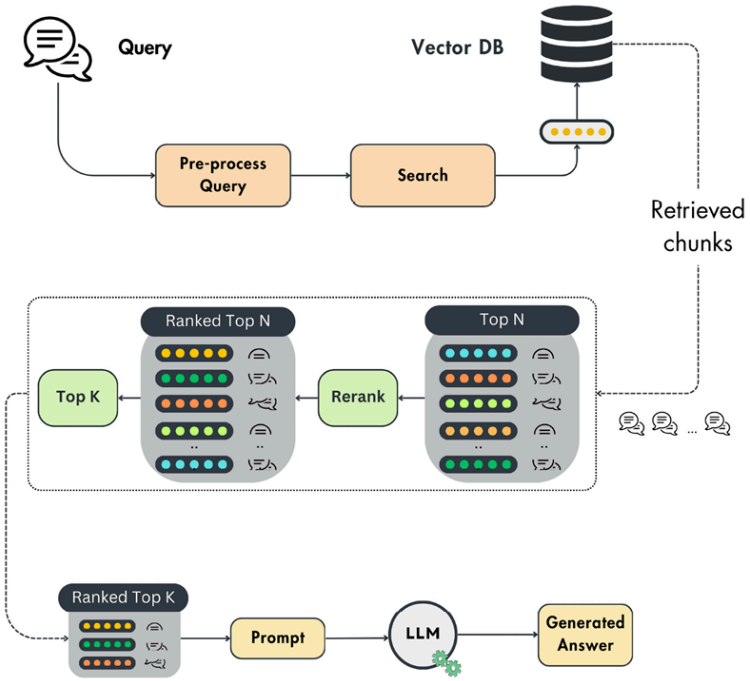

Figure 4.8: The re-ranking algorithm

To summarize, the primary goal of these optimizations is to enhance the RAG algorithm at three key stages: pre-retrieval, retrieval, and post-retrieval. This involves preprocessing data for improved vector indexing, adjusting user queries for more accurate searches, enhancing the embedding model, utilizing classic filtering DB operations, and removing noisy data. By keeping these goals in mind, you can effectively optimize your RAG workflow for data processing and retrieval

Exploring the LLM Twin's RAG feature pipeline architecture

Now that you have a strong intuition and understanding of RAG and its workings, we will continue exploring our particular LLM Twin use case. The goal is to provide a hands-on end-to-end example to solidify the theory presented in this chapter.

Any RAG system is split into two independent components:

- The **ingestion pipeline** takes in raw data, cleans, chunks, embeds, and loads it into a vector DB.
- The **inference pipeline** queries the vector DB for relevant context and ultimately generates an answer by levering an LLM.

In this chapter, we will focus on implementing the RAG ingestion pipeline, and in *Chapter 9*, we will continue developing the inference pipeline.

With that in mind, let's have a quick refresher on the problem we are trying to solve and where we get our raw data. Remember that we are building an end-to-end ML system. Thus, all the components talk to each other through an interface (or a contract), and each pipeline has a single responsibility. In our case, we ingest raw documents, preprocess them, and load them into a vector DB.

The problem we are solving

As presented in the previous chapter, this book aims to show you how to build a production-ready LLM Twin backed by an end-to-end ML system. In this chapter specifically, we want to design a RAG feature pipeline that takes raw social media data (e.g., articles, code repositories, and posts) from our MongoDB data warehouse. The text of the raw documents will be cleaned, chunked, embedded, and ultimately loaded to a feature store. As discussed in *Chapter 1*, we will implement a logical feature store using ZenML artifacts and a Qdrant vector DB.

As we want to build a fully automated feature pipeline, we want to sync the data warehouse and logical feature store. Remember that, at inference time, the context used to generate the answer is retrieved from the vector DB. Thus, the speed of synchronization between the data warehouse and the feature store will directly impact the accuracy of our RAG algorithm.

Another key consideration is how to automate the feature pipeline and integrate it with the rest of our ML system. Our goal is to minimize any desynchronization between the two data storages, as this could potentially compromise the integrity of our system.

To conclude, we must design a feature pipeline that constantly syncs the data warehouse and logical feature store while processing the data accordingly. Having the data in a feature store is critical for a production-ready ML system. The LLM Twin inference pipeline will query it for RAG, while the training pipeline will consume tracked and versioned fine-tuning datasets from it.

The feature store

The **feature store** will be the **central access point** for all the features used within the training and inference pipelines. The training pipeline will use the cleaned data from the feature store (stored as artifacts) to fine-tune LLMs. The inference pipeline will query the vector DB for chunked documents for RAG. That is why we are designing a feature pipeline and not only a RAG ingestion pipeline. In practice, the feature pipeline contains multiple subcomponents, one of which is the RAG logic.

Remember that the feature pipeline is mainly used as a mind map to navigate the complexity of ML systems. It clearly states that it takes raw data as input and then outputs features and optional labels, which are stored in the feature store. Thus, a good intuition is to consider that all the logic between the data warehouse and the feature store goes into the feature pipeline namespace, consisting of one or more sub-pipelines. For example, we will implement another pipeline that takes in cleaned data, processes it into instruct datasets, and stores it in artifacts; this also sits under the feature pipeline umbrella as the artifacts are part of the logical feature store. Another example would be implementing a data validation pipeline on top of the raw data or computed features.

Another important observation to make is that text data stored as strings are not considered features if you follow the standard conventions. A feature is something that is fed directly into the model. For example, we would have to tokenize the instruct datasets or chunked documents to be considered features. Why? Because the tokens are fed directly to the model and not the sentences as strings. Unfortunately, this makes the system more complex and unflexible. Thus, we will do the tokenization at runtime. But this observation is important to understand as it's a clear example that you don't have to be too rigid about the feature/training/inference (FTI) architecture. You have to take it and adapt it to your own use case.

Where does the raw data come from?

As a quick reminder, all the raw documents are stored in a MongoDB data warehouse. The data warehouse is populated by the data collection ETL pipeline presented in *Chapter 3*. The ETL pipeline crawls various platforms such as Medium and Substack, standardizes the data, and loads it into MongoDB. Check out *Chapter 3* for more details on this topic.

Designing the architecture of the RAG feature pipeline

The last step is to architect and go through the design of the RAG feature pipeline of the LLM Twin application. We will use a batch design scheduled to poll data from the MongoDB data warehouse, process it, and load

it to a Qdrant vector DB. The first question to ask ourselves is, “Why a batch pipeline?”

But before answering that, let’s quickly understand how a batch architecture works and behaves relative to a streaming design.

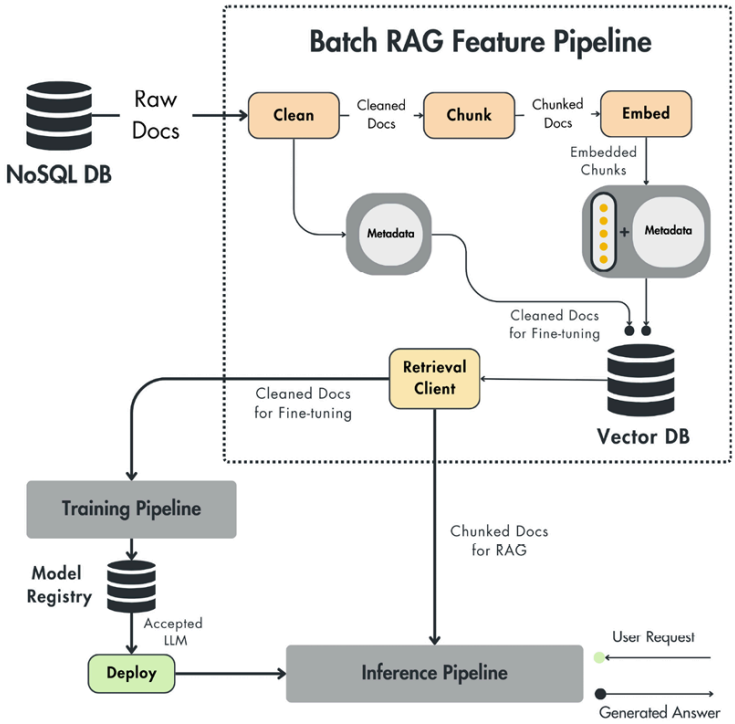

Figure 4.9: The architecture of the LLM Twin’s RAG feature pipeline

Batch pipelines

A batch pipeline in data systems refers to a data processing method where data is collected, processed, and stored in predefined intervals and larger volumes, also known as “batches”. This approach differs from real-time or streaming data processing, where data is processed continuously as it arrives. This is what happens in a batch pipeline:

1. **Data collection**: Data is collected from various sources and stored until sufficient amounts are accumulated for processing. This can include data from DBs, logs, files, and other sources.
2. **Scheduled processing**: Data processing is scheduled at regular intervals, for example, hourly or daily. During this time, the collected data is processed in bulk. This can involve data cleansing, transformation, aggregation, and other operations.
3. **Data loading**: After processing, the data is loaded into the target system, such as a DB, data warehouse, data lake, or feature store. This processed data is then available for analysis, querying, or further processing.

Batch pipelines are particularly useful when dealing with large volumes of data that do not require immediate processing. They offer several advantages, including:

- **Efficiency**: Batch processing can handle large volumes of data more efficiently than real-time processing, allowing for optimized resource allocation and parallel processing.

- **Complex processing**: Batch pipelines can perform complex data transformations and aggregations that might be too resource-intensive for real-time processing.
- **Simplicity**: Batch processing systems' architectures are often simpler than those of real-time systems, making them easier to implement and maintain.

Batch versus streaming pipelines

When implementing feature pipelines, you have two main design choices: batch and streaming. Thus, it is worthwhile to see the difference between the two and understand why we chose a batch architecture over a streaming one for our LLM Twin use case.

You can effortlessly write a dedicated chapter on streaming pipelines, which suggests its complexity over a batch design. However, as streaming architectures become increasingly popular, one must have an intuition of how they work to choose the best option for your application.

The core elements of streaming applications are a distributed event streaming platform such as Apache Kafka or Redpanda to store events from multiple clients and a streaming engine such as Apache Flink or Bytewax to process the events. To simplify your architecture, you can swap your event streaming platform with queues, such as RabbitMQ, to store the events until processed. *Table 4.1* compares batch and streaming pipelines based on multiple criteria such as processing schedule and complexity:

Aspect	Batch pipeline	Streaming pipeline
Processing schedule	Processes data at regular intervals (e.g., every minute, hourly, daily).	Processes data continuously, with minimal latency.
Efficiency	Handles large volumes of data more efficiently, optimizing resource allocation and parallel processing.	Handles single data points, providing immediate insights and updates, allowing for rapid response to changes.
Processing complexity	Capable of performing complex data transformations and aggregations.	Designed to handle high-velocity data streams with low latency.
Use cases	Suitable for scenarios where immediate data processing is not critical. Commonly used in	Ideal for applications requiring real-time analytics, features, monitoring, and event-driven architectures.

	data warehousing, reporting, ETL processes, and feature pipelines.	
System complexity	Compared to streaming pipelines, systems are generally simpler to implement and maintain.	More complex to implement and maintain due to the need for low-latency processing, fault tolerance, and scalability. The tooling is also more advanced and complicated.

Table 4.1: Batch versus streaming pipelines

For example, streaming pipelines are extremely powerful in social media recommender systems like TikTok. When using social media, user behavior changes frequently. A typical scenario is that you want to relax at a certain point in time and mostly look at videos of puppies. Still, after 15 minutes, you get bored and want something more serious, such as educative content or news. This means the recommender system has to capture these behavior changes without delay to keep you engaged. As the transition between interests is cyclical and not predictable, you can't use a batch pipeline that runs every 30 minutes or every hour to generate more content. You can run it every minute to create new content, but, at the same time, it will result in unnecessary costs, as most predictions will not be consumed. By implementing a streaming pipeline, you update the features of specific users in real time, which are then passed to a chain of models that predict the new recommendations.

Streaming architectures are also the backbone of real-time fraud detection algorithms, such as those used at Stripe or PayPal. In this context, it's critical to identify potentially fraudulent transactions as they occur, not after a few minutes or hours as a batch pipeline would process them. The same urgency applies to high-frequency trading platforms that make stock predictions based on the constant influx of market data, enabling traders to make decisions within milliseconds.

On the other hand, you can use a batch architecture for an offline recommender system. For example, when implementing one for an e-commerce or streaming platform, you don't need the system to be so reactive, as the user's behavior rarely changes. Thus, updating the recommendations periodically, such as every night, based on historical user behavior data using a batch pipeline is easier to implement and cheaper.

Another popular example of batch pipelines is the ETL design used to extract, transform, and load data for different use cases. The ETL design is widespread in data pipelines used to move data from one DB to another. Some practical use cases include aggregating data for analytics, where you have to extract data from multiple sources, aggregate it, and load it to a data warehouse connected to a dashboard. The analytics domains can be widespread, from e-commerce and marketing to finance and research.

The data collection pipeline used in the LLM Twin use case is another example of an ETL pipeline that extracts data from the internet, structures it, and loads it into a data warehouse for future processing.

Along with prediction or feature freshness, another disadvantage of batch pipelines over streaming ones is that you usually make redundant predictions. Let's take the example of a recommender system for a streaming platform like Netflix. Every night, you make the predictions for all users. There is a significant chance that a large chunk of users won't log in that day. Also, users usually don't browse all the recommendations but stick to the first ones. Thus, only a portion of predictions are used, wasting computing power on all the others.

That's why a popular strategy is to start with a batch architecture, as it's faster and easier to implement. After the product is in place, you gradually move to a streaming design to reduce costs and improve the user experience.

To conclude, we have used a batch architecture (and not a streaming one) to implement the LLM Twin's feature pipeline for the following reasons:

- **Does not require immediate data processing**: Even if syncing the data warehouse and feature store is critical for an accurate RAG system, a delay of a few minutes is acceptable. Thus, we can schedule the batch pipeline to run every minute, constantly syncing the two data storages. This technique works because the data volume is small. The whole data warehouse will have only thousands of records, not millions or billions. Hence, we can quickly iterate through them and sync the two DBs.
- **Simplicity**: As stated earlier, implementing a streaming pipeline is two times more complex. In the real world, you want to keep your system as simple as possible, making it easier to understand, debug, and maintain. Also, simplicity usually translates to lower infrastructure and development costs.

In *Figure 8.10*, we compare what tools you can use based on your architecture (streaming versus batch) and the quantity of data you have to process (small versus big data). In our use case, we are in the smaller data and batch quadrant, where we picked a combination of vanilla Python and generative AI tools such as LangChain, Sentence Transformers, and Unstructured.

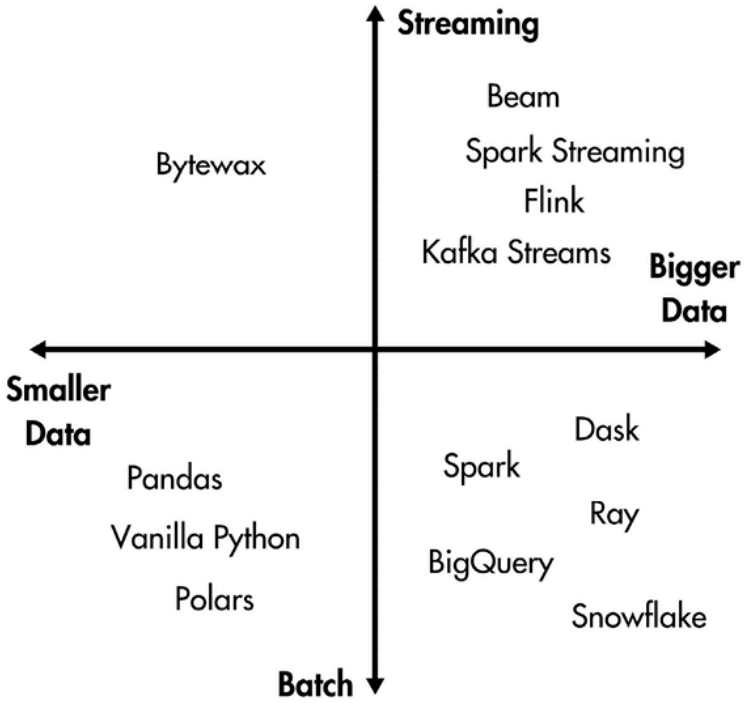

Figure 4.10: Tools on the streaming versus batch and smaller versus bigger data spectrum

In the *Change data capture: syncing the data warehouse and feature store* section later in this chapter, we will discuss when switching from a batch architecture to a streaming one makes sense.

Core steps

Most of the RAG feature pipelines are composed of five core steps. The one implemented in the LLM Twin architecture makes no exception. Thus, you can quickly adapt this pattern for other RAG applications, but here is what the LLM Twin's RAG feature pipeline looks like:

1. **Data extraction**: Extract the latest articles, code repositories, and posts from the MongoDB data warehouse. At the extraction step, you usually aggregate all the data you need for processing.
2. **Cleaning**: The data from the data warehouse is standardized and partially clean, but we have to ensure that the text contains only useful information, is not duplicated, and can be interpreted by the embedding model. For example, we must clean and normalize all non-ASCII characters before passing the text to the embedding model. Also, to keep the information semantically dense, we decided to replace all the URLs with placeholders and remove all emojis. The cleaning step is more art than **science**. Hence, after you have the first iteration with an evaluation mechanism in place, you will probably reiterate and improve it.
3. **Chunking**: You must adopt various chunking strategies based on each data category and embedding model. For example, when working with code repositories, you want the chunks broader, whereas when working with articles, you want them narrower or scoped at the paragraph level. Depending on your data, you must decide if you split your document based on the chapter, section, paragraph, sentence, or just a fixed window size. Also, you have to ensure that the chunk size doesn't exceed the maximum input size of the embedding model. That is why you usually chunk a document based on your data structure and the maximum input size of the model.

4. **Embedding**: You pass each chunk individually to an embedding model of your choice. Implementation-wise, this step is usually the simplest, as tools such as SentenceTransformer and Hugging Face provide high-level interfaces for most embedding models. As explained in the *What are embeddings?* section of this chapter, at this step, the most critical decisions are to decide what model to use and whether to fine-tune it or not. For example, we used an `"all-mpnet-base-v2"` embedding model from *SentenceTransformer*, which is relatively tiny and runs on most machines. However, we provide a configuration file where you can quickly configure the embedding model with something more powerful based on the state of the art when reading this book. You can quickly find other options on the MTEB on Hugging Face (https://huggingface.co/spaces/mteb/leaderboard).
5. **Data loading**: The final step combines the embedding of a chunked document and its metadata, such as the author and the document ID, content, URL, platform, and creation date. Ultimately, we wrap the vector and the metadata into a structure compatible with Qdrant and push it to the vector DB. As we want to use Qdrant as the single source of truth for the features, we also push the cleaned documents (before chunking) to Qdrant. We can push data without vectors, as the metadata index of Qdrant behaves like a NoSQL DB. Thus, pushing metadata without a vector attached to it is like using a standard NoSQL engine.

Change data capture: syncing the data warehouse and feature store

As highlighted a few times in this chapter, data is constantly changing, which can result in DBs, data lakes, data warehouses, and feature stores getting out of sync. **Change data capture** (**CDC**) is a strategy that allows you to optimally keep two or more data storage types in sync without computing and I/O overhead. It captures any CRUD operation done on the source DB and replicates it on a target DB. Optionally, you can add preprocessing steps in between the replication.

The syncing issues also apply when building a feature pipeline. One key design choice concerns how to sync the data warehouse with the feature store to have data fresh enough for your particular use case.

In our LLM Twin use case, we chose a naïve approach out of simplicity. We implemented a batch pipeline that is triggered periodically or manually. It reads all the raw data from the data warehouse, processes it in batches, and inserts new records or updates old ones from the Qdrant vector DB. This works fine when you are working with a small number of records, at the order of thousands or tens of thousands. But our naïve approach raises the following questions:

- What happens if the data suddenly grows to millions of records (or higher)?
- What happens if a record is deleted from the data warehouse? How is this reflected in the feature store?
- What if we want to process only the new or updated items from the data warehouse and not all of them?

Fortunately, the CDC pattern can solve all of these issues. When implementing CDC, you can take multiple approaches, but all of them use ei-

ther a push or pull strategy:

- **Push:** The source DB is the primary driver in the push approach. It actively identifies and transmits data modifications to target systems for processing. This method ensures near-instantaneous updates at the target, but data loss can occur if target systems are inaccessible. To mitigate this, a messaging system is typically employed as a buffer.
- **Pull:** The pull method assigns a more passive role to the source DB, which only records data changes. Target systems periodically request these changes and handle updates accordingly. While this approach lightens the load on the source, it introduces a delay in data propagation. A messaging system is again essential to prevent data loss during periods of target system unavailability.

In summary, the push method is ideal for applications demanding immediate data access, whereas the pull method is better suited for large-scale data transfers where real-time updates aren't critical. With that in mind, there are different methods to detect changes in data. Thus, let's list the main CDC patterns that are used in the industry:

- **Timestamp-based**: The approach involves adding a modification time column to DB tables, usually called `LAST_MODIFIED` or `LAST_UPDATED`. Downstream systems can query this column to identify records that have been updated since their last check. While simple to implement, this method is limited to tracking changes, not deletions, and imposes performance overhead due to the need to scan entire tables.
- **Trigger-based**: The trigger-based approach utilizes DB triggers to automatically record data modifications in a separate table upon INSERT, UPDATE, or DELETE operations, often known as the event table. This method provides comprehensive change tracking but can impact the DB performance due to the additional write operations involved for each event.
- **Log-based**: DBs maintain transaction logs to record all data modifications, including timestamps. Primarily used for recovery, these logs can also be leveraged to propagate changes to target systems in real time. This approach minimizes the performance impact on the source DB. As a huge advantage, it avoids additional processing overhead on the source DB, captures all data changes, and requires no schema modification. But on the opposite side, it lacks standardized log formats, leading to vendor-specific implementations.

For more details on CDC, I recommend *What is Change Data Capture?* from Confluent's blog: https://www.confluent.io/en-gb/learn/change-data-capture/.

With these CDC techniques in mind, we could quickly implement a pull timestamp-based strategy in our RAG feature pipeline to sync the data warehouse and feature store more optimally when the data grows. Our implementation is still pull-based but doesn't check any last updated field in the source DB; it just pulls everything from the data warehouse.

However, the most popular and optimal technique in the industry is the log-based one. It doesn't add any I/O overhead to the source DB, has low latency, and supports all CRUD operations. The biggest downside is its de-

velopment complexity, which requires a queue to capture all the CRUD events and a streaming pipeline to process them.

As this is an LLM book and not a data engineering one, we wanted to keep things simple, but it's important to know that these techniques exist, and you can always upgrade your current implementation when it doesn't fit your application requirements anymore.

Why is the data stored in two snapshots?

We store two snapshots of our data in the logical feature store:

- **After the data is cleaned**: For fine-tuning LLMs
- **After the documents are chunked and embedded**: For RAG

Why did we design it this way? Remember that the features should be accessed solely from the feature store for training and inference. Thus, this adds consistency to our design and makes it cleaner.

Also, storing the data cleaned specifically for our fine-tuning and embedding use case in the MongoDB data warehouse would have been an antipattern. The data from the warehouse is shared all across the company. Thus, processing it for a specific use case is not good practice. Imagine another summarization use case where we must clean and preprocess the data differently. We must create a new "Cleaned Data" table prefixed with the use case name. We have to repeat that for every new use case. Therefore, to avoid having a spaghetti data warehouse, the data from the data warehouse is generic and is modeled to specific applications only in downstream components, which, in our case, is the feature store.

Ultimately, as we mentioned in the *Core steps* section, you can leverage the metadata index of a vector DB as a NoSQL DB. Based on these factors, we decided to keep the cleaned data in Qdrant, along with the chunked and embedded versions of the documents.

As a quick reminder, when operationalizing our LLM Twin system, the create instruct dataset pipeline, explained in *Chapter 5*, will read the cleaned documents from Qdrant, process them, and save them under a versioned ZenML artifact. The training pipeline requires a dataset and not plain documents. This is a reminder that our logical feature store comprises the Qdrant vector DB for online serving and ZenML artifacts for offline training.

Orchestration

ZenML will orchestrate the batch RAG feature pipeline. Using ZenML, we can schedule it to run on a schedule, for example, every hour, or quickly manually trigger it. Another option is to trigger it after the ETL data collection pipeline finishes.

By orchestrating the feature pipeline and integrating it into ZenML (or any other orchestration tool), we can operationalize the feature pipeline with the end goal of **continuous training (CT)**.

We will go into all the details of orchestration, scheduling, and CT in *Chapter 11*.

Implementing the LLM Twin's RAG feature pipeline

The last step is to review the LLM Twin's RAG feature pipeline code to see how we applied everything we discussed in this chapter. We will walk you through the following:

- ZenML code
- Pydantic domain objects
- A custom **object-vector mapping** (**OVM**) implementation
- The cleaning, chunking, and embedding logic for all our data categories

We will take a top-down approach. Thus, let's start with the Settings class and ZenML pipeline.

Settings

We use Pydantic Settings (https://docs.pydantic.dev/latest/concepts/pydantic_settings/) to define a global Settings class that loads sensitive or non-sensitive variables from a `.env` file. This approach also gives us all the benefits of Pydantic, such as type validation. For example, if we provide a string for the `QDRANT_DATABASE_PORT` variable instead of an integer, the program will crash. This behavior makes the whole application more deterministic and reliable.

Here is what the `Settings` class looks like with all the variables necessary to build the RAG feature pipeline:

```
from pydantic import BaseSettings
class Settings(BaseSettings):
    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"
    … # Some other settings…
    # RAG
    TEXT_EMBEDDING_MODEL_ID: str = "sentence-transformers/all-MiniLM-L6-v2"
    RERANKING_CROSS_ENCODER_MODEL_ID: str = "cross-encoder/ms-marco-MiniLM-L-4-v2"
    RAG_MODEL_DEVICE: str = "cpu"
    # QdrantDB Vector DB
    USE_QDRANT_CLOUD: bool = False
    QDRANT_DATABASE_HOST: str = "localhost"
    QDRANT_DATABASE_PORT: int = 6333
    QDRANT_CLOUD_URL: str = "str"
    QDRANT_APIKEY: str | None = None
    … # More settings…
settings = Settings()
```

As stated in the internal Config class, all the variables have default values or can be overridden by providing a `.env` file.

ZenML pipeline and steps

The ZenML pipeline is the entry point for the RAG feature engineering pipeline. It reflects the five core phases of RAG ingestion code: extracting raw documents, cleaning, chunking, embedding, and loading them to the logical feature store. The calls within the `feature_engineering()` function are ZenML steps, representing a single execution unit perform-

ing the five phases of RAG. The code is available in the GitHub repository at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/pipelines/feature_engineering.py:

```
from zenml import pipeline
from llm_engineering.interfaces.orchestrator.steps import feature_engineering as fe
@pipeline
def feature_engineering(author_full_names: list[str]) -> None:
    raw_documents = fe_steps.query_data_warehouse(author_full_names)
    cleaned_documents = fe_steps.clean_documents(raw_documents)
     last_step_1 = fe_steps.load_to_vector_db(cleaned_documents)
    embedded_documents = fe_steps.chunk_and_embed(cleaned_documents)
    last_step_2 = fe_steps.load_to_vector_db(embedded_documents)
    return [last_step_1.invocation_id, last_step_2.invocation_id]
```

Figure 4.11 shows how multiple feature engineering pipeline runs look in ZenML's dashboard.

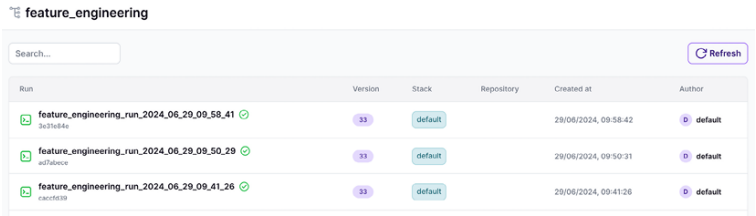

Figure 4.11: Feature pipeline runs in the ZenML dashboard

Figure 8.12 shows the DAG of the RAG feature pipeline, where you can follow all the pipeline steps and their output artifacts. Remember that whatever is returned from a ZenML step is automatically saved as an artifact, stored in ZenML's artifact registry, versioned, and shareable across the application.

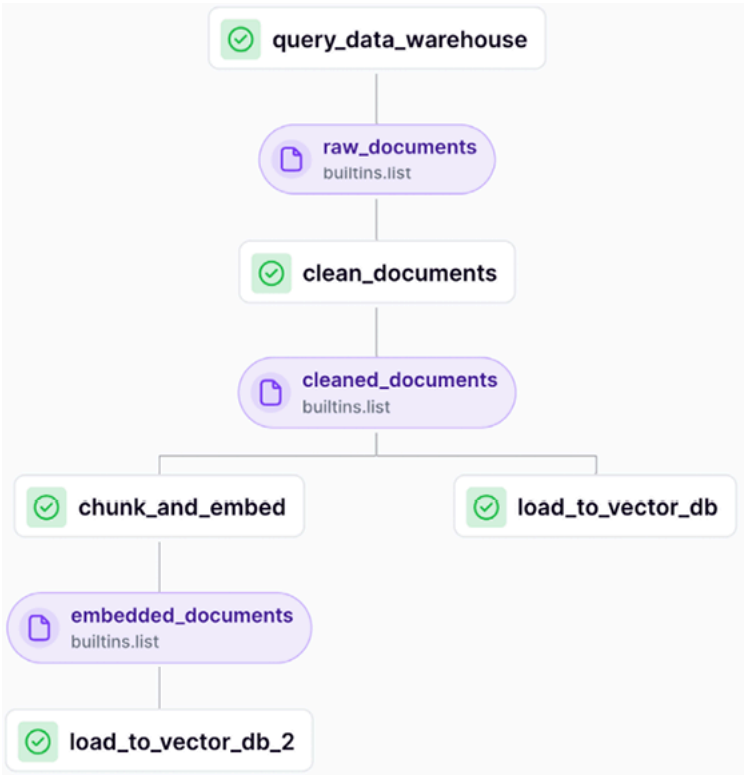

Figure 4.12: Feature pipeline DAG in the ZenML dashboard

The final puzzle piece is understanding how to configure the RAG feature pipeline dynamically. All its available settings are exposed as function parameters. Here, we need only a list of author's names, as seen in the function's signature: `feature_engineering(author_full_names: list[str])`. We inject a YAML configuration file at runtime that contains all the necessary values based on different use cases. For example, the following configuration includes a list of all the authors of this book as we want to populate the feature store with data from all of us (available in the GitHub repository at `configs/feature_engineering.yaml`):

```
parameters:
  author_full_names:
    - Alex Vesa
    - Maxime Labonne
    - Paul Iusztin
```

The beauty of this approach is that you don't have to modify the code to configure the feature pipeline with different input values. You have to provide a different configuration file when running it, as follows:

```
feature_engineering.with_options(config_path="…/feature_engineering.yaml")()
```

You can either hardcode the path to the config file or provide the `config_path` from the CLI, which allows you to modify the pipeline's configuration between different runs. Out of simplicity, we hard-coded the configuration file. Thus, we can call the feature engineering pipeline calling the `run.py` script as follows:

```
python -m tools.run --no-cache --run-feature-engineering
```

However, you can easily add another CLI argument to pass the `config_path` variable. Also, you can run the feature pipeline using the following `poe` command:

```
poetry poe run-feature-engineering-pipeline
```

Let's move forward to the ZenML steps and sequentially zoom in on all of them. The source code for all the feature engineering pipeline steps is available on GitHub at `"steps/feature_engineering"`. We will begin with the first step, which involves querying the data warehouse for new content to process into features.

Querying the data warehouse

The first thing to notice is that a step is a Python function decorated with `@step`, similar to how a ZenML pipeline works. The function below takes as input a list of authors' full names and performs the following core steps:

- It attempts to get or create a `UserDocument` instance using the first and last names, appending this instance to the `authors` list. If the user

doesn't exist, it throws an error.

- It fetches all the raw data for the user from the data warehouse and extends the `documents` list to include these user documents.
- Ultimately, it computes a descriptive metadata dictionary logged and tracked in ZenML.

```
… # other imports
from zenml import get_step_context, step
@step
def query_data_warehouse(
    author_full_names: list[str],
) -> Annotated[list, "raw_documents"]:
    documents = []
    authors = []
    for author_full_name in author_full_names:
        logger.info(f"Querying data warehouse for user: {author_full_name}")
        first_name, last_name = utils.split_user_full_name(author_full_name)
        logger.info(f"First name: {first_name}, Last name: {last_name}")
        user = UserDocument.get_or_create(first_name=first_name, last_name=last_nar
        authors.append(user)
        results = fetch_all_data(user)
        user_documents = [doc for query_result in results.values() for doc in query
        documents.extend(user_documents)
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="raw_documents", metadata=_get_met
    return documents
```

The fetch function leverages a thread pool that runs each query on a different thread. As we have multiple data categories, we have to make a different query for the articles, posts, and repositories, as they are stored in different collections. Each query calls the data warehouse, which is bounded by the network I/O and data warehouse latency, not by the machine's CPU. Thus, by moving each query to a different thread, we can parallelize them. Ultimately, instead of adding the latency of each query as the total timing, the time to run this fetch function will be the max between all the calls.

Using threads to parallelize I/O-bounded calls is good practice in Python, as they are not locked by the Python **Global Interpreter Lock** (**GIL**). In contrast, adding each call to a different process would add too much overhead, as a process takes longer to spin off than a thread.

In Python, you want to parallelize things with processes only when the operations are CPU or memory-bound because the GIL affects them. Each process has a different GIL. Thus, parallelizing your computing logic, such as processing a batch of documents or images already loaded in memory, isn't affected by Python's GIL limitations.

```
def fetch_all_data(user: UserDocument) -> dict[str, list[NoSQLBaseDocument]]:
    user_id = str(user.id)
    with ThreadPoolExecutor() as executor:
        future_to_query = {
            executor.submit(__fetch_articles, user_id): "articles",
            executor.submit(__fetch_posts, user_id): "posts",
            executor.submit(__fetch_repositories, user_id): "repositories",
        }
        results = {}
        for future in as_completed(future_to_query):
            query_name = future_to_query[future]
```

```
            try:
                results[query_name] = future.result()
            except Exception:
                logger.exception(f"'{query_name}' request failed.")
                results[query_name] = []
    return results
```

The `_get_metadata()` function takes the list of queried documents and authors and counts the number of them relative to each data category:

```
def _get_metadata(documents: list[Document]) -> dict:
    metadata = {
        "num_documents": len(documents),
    }
    for document in documents:
        collection = document.get_collection_name()
        if collection not in metadata:
            metadata[collection] = {}
        if "authors" not in metadata[collection]:
            metadata[collection]["authors"] = list()
        metadata[collection]["num_documents"] = metadata[collection].get("num_docur
        metadata[collection]["authors"].append(document.author_full_name)
    for value in metadata.values():
        if isinstance(value, dict) and "authors" in value:
            value["authors"] = list(set(value["authors"]))
    return metadata
```

We will expose this metadata in the ZenML dashboard to quickly see some statistics on the loaded data. For example, in *Figure 4.13*, we accessed the metadata tab of the `query_data_warehouse()` step, where you can see that, within that particular run of the feature pipeline, we loaded 76 documents from three authors. This is extremely powerful for monitoring and debugging batch pipelines.

You can always extend it with anything that makes sense for your use case.

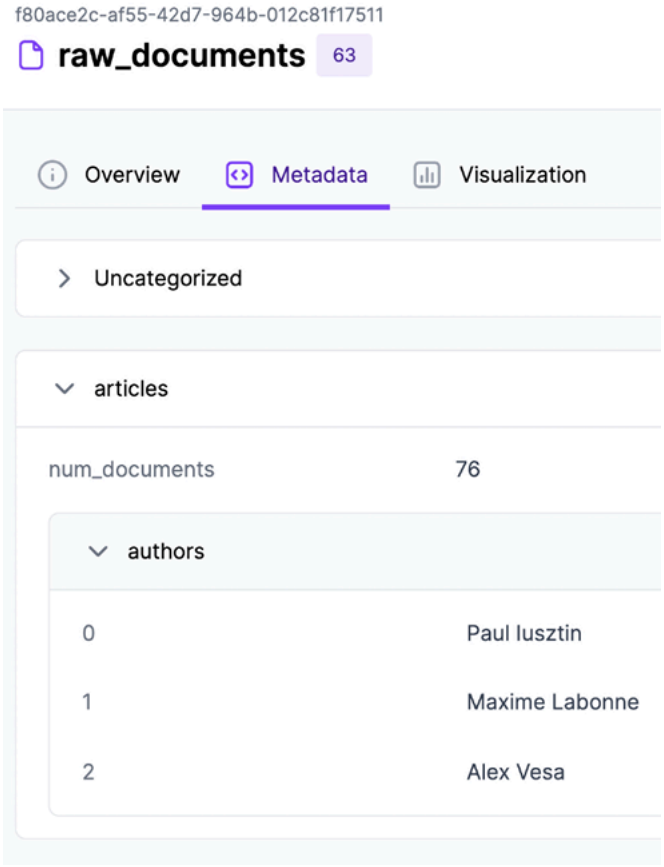

Figure 4.13: Metadata of the "query the data warehouse" ZenML step

Cleaning the documents

In the cleaning step, we iterate through all the documents and delegate all the logic to a `CleaningDispatcher` who knows what cleaning logic to apply based on the data category. Remember that we want to apply, or have the ability to apply in the future, different cleaning techniques on articles, posts, and code repositories.

```
@step
def clean_documents(
    documents: Annotated[list, "raw_documents"],
) -> Annotated[list, "cleaned_documents"]:
    cleaned_documents = []
    for document in documents:
        cleaned_document = CleaningDispatcher.dispatch(document)
        cleaned_documents.append(cleaned_document)
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="cleaned_documents", metadata=_ge
    return cleaned_documents
```

The computed metadata is similar to what we logged in the `query_data_warehouse()` step. Thus, let's move on to chunking and embedding.

Chunk and embed the cleaned documents

Similar to how we cleaned the documents, we delegate the chunking and embedding logic to a dispatcher who knows how to handle each data category. Note that the chunking dispatcher returns a list instead of a single

object, which makes sense as the document is split into multiple chunks. We will dig into the dispatcher in the "The dispatcher layer" section of this chapter.

```
@step
def chunk_and_embed(
    cleaned_documents: Annotated[list, "cleaned_documents"],
) -> Annotated[list, "embedded_documents"]:
    metadata = {"chunking": {}, "embedding": {}, "num_documents": len(cleaned_docur
    embedded_chunks = []
    for document in cleaned_documents:
        chunks = ChunkingDispatcher.dispatch(document)
        metadata["chunking"] = _add_chunks_metadata(chunks, metadata["chunking"])
        for batched_chunks in utils.misc.batch(chunks, 10):
            batched_embedded_chunks = EmbeddingDispatcher.dispatch(batched_chunks)
            embedded_chunks.extend(batched_embedded_chunks)
    metadata["embedding"] = _add_embeddings_metadata(embedded_chunks, metadata["emb
    metadata["num_chunks"] = len(embedded_chunks)
    metadata["num_embedded_chunks"] = len(embedded_chunks)
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="embedded_documents", metadata=met
    return embedded_chunks
```

In *Figure 4.14*, you can see the metadata of the chunking and embedding ZenML step. For example, you can quickly understand that we transformed 76 documents into 2,373 chunks, or the properties we used for chunking articles, such as a `chunk_size` of 500 and a `chunk_overlap` of **50**.

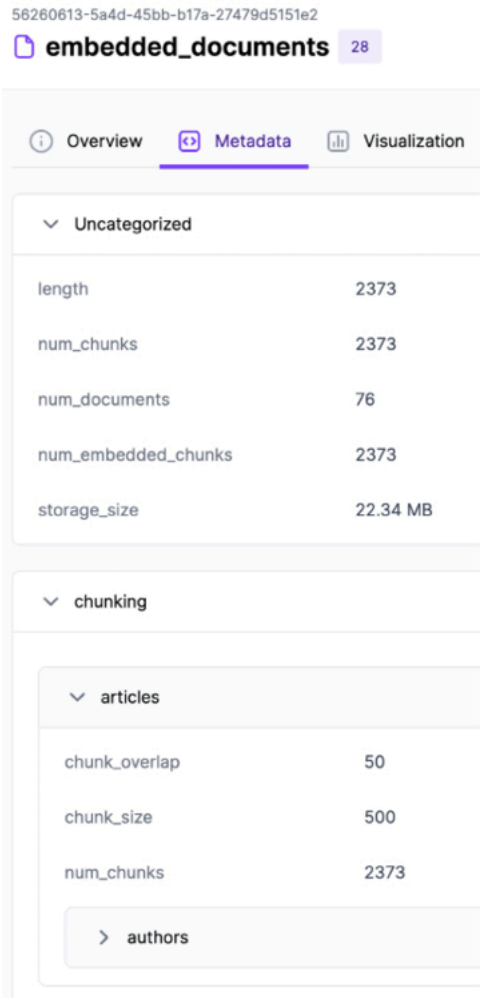

Figure 4.14: Metadata of the embedding and chunking ZenML step, detailing the uncategorized and chunking dropdowns

In Figure 4.15, the rest of the ZenML metadata from the embedding and chunking step details the embedding model and its properties used to compute the vectors.

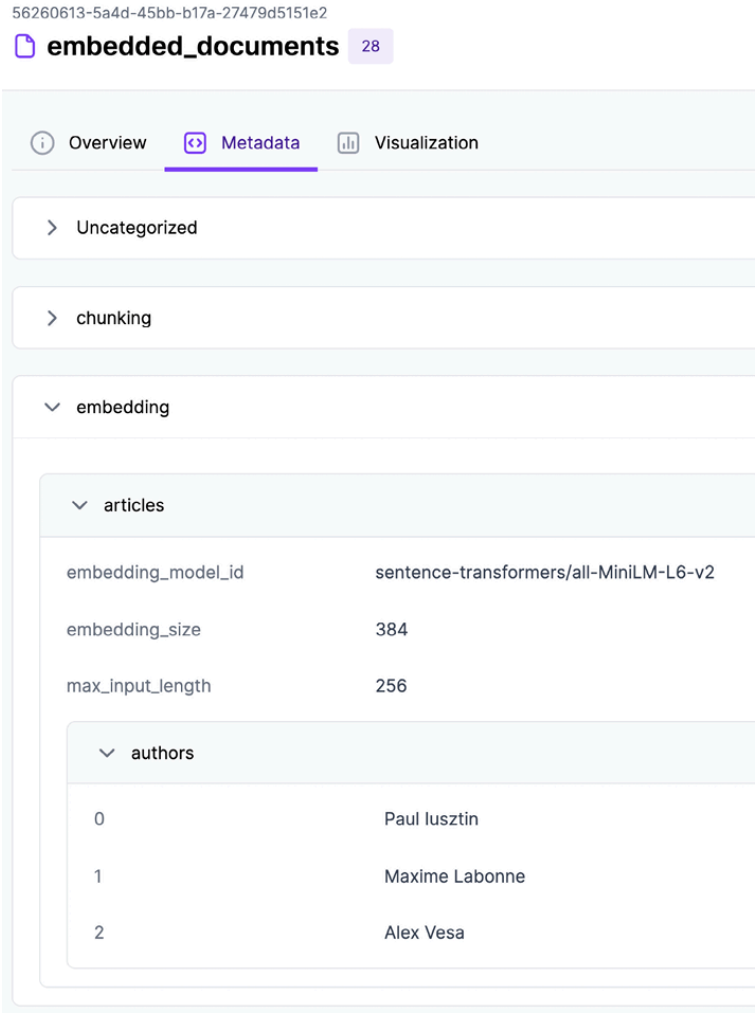

Figure 4.15: Metadata of the embedding and chunking ZenML step, detailing the embedding dropdown

As ML systems can break at any time while in production due to drifts or untreated use cases, leveraging the metadata section to monitor the ingested data can be a powerful tool that will save debugging days, translating to tens of thousands of dollars or more for your business.

Loading the documents to the vector DB

As each article, post, or code repository sits in a different collection inside the vector DB, we have to group all the documents based on their data category. Then, we load each group in bulk in the Qdrant vector DB:

```
@step
def load_to_vector_db(
    documents: Annotated[list, "documents"],
) -> None:
    logger.info(f"Loading {len(documents)} documents into the vector database.")
```

```
        grouped_documents = VectorBaseDocument.group_by_class(documents)
        for document_class, documents in grouped_documents.items():
            logger.info(f"Loading documents into {document_class.get_collection_name()}")
            for documents_batch in utils.misc.batch(documents, size=4):
                try:
                    document_class.bulk_insert(documents_batch)
                except Exception:
                    return False
        return True
```

Pydantic domain entities

Before investigating the dispatchers, we must understand the domain objects we work with. To some extent, in implementing the LLM Twin, we are following the **domain-driven design** (**DDD**) principles, which state that domain entities are the core of your application. Thus, before proceeding, it's important to understand the hierarchy of the domain classes we are working with.

The code for the domain entities is available on GitHub at https://github.com/PacktPublishing/LLM-Engineering/tree/main/llm_engineering/domain.

We used Pydantic to model all our domain entities. When we wrote the book, choosing Pydantic was a no-brainer, as it is the go-to Python package for writing data structures with out-of-the-box type validation. As Python is a dynamically typed language, using Pydantic for type validation at runtime makes your system order of times more robust, as you can be sure that you are always working with the right type of data.

The domain of our LLM Twin application is split into two dimensions:

- **The data category**: Post, article, and repository
- **The state of the data**: Cleaned, chunked, and embedded

We decided to create a base class for each state of the document, resulting in having the following base abstract classes:

- `class CleanedDocument(VectorBaseDocument, ABC)`
- `class Chunk(VectorBaseDocument, ABC)`
- `class EmbeddedChunk(VectorBaseDocument, ABC)`

Note that all of them inherit the `VectorBaseDocument` class, which is our custom **OVM** implementation, which we will explain in the next section of this chapter. Also, it inherits from ABC, which makes the class abstract. Thus, you cannot initialize an object out of these classes; you may only inherit from them. That is why base classes are always marked as abstract.

Each base abstract class from above (which models the state) will have a subclass that adds the data category dimension. For example, the `CleanedDocument` class will have the following subclasses:

- `class CleanedPostDocument(CleanedDocument)`
- `class CleanedArticleDocument(CleanedDocument)`
- `class CleanedRepositoryDocument(CleanedDocument)`

As we can see in *Figure 8.16*, we will repeat the same logic for the `Chunk` and `EmbeddedChunk` base abstract classes. We will implement a specific document class for each data category and state combination, resulting in nine types of domain entities. For example, when ingesting a raw document, the cleaning step will yield a `CleanedArticleDocument` instance, the chunking step will return a list of `ArticleChunk` objects, and the embedding operation will return `EmbeddedArticleChunk` instances that encapsulate the embedding and all the necessary metadata to ingest in the vector DB.

The same will happen for the posts and repositories.

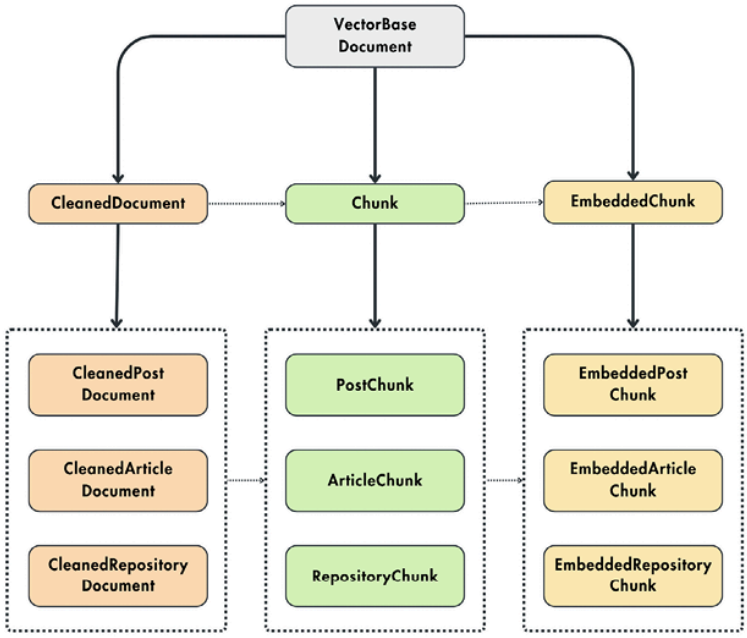

Figure 4.16: Domain entities class hierarchy and their interaction

We chose this design because the list of states will rarely change, and we want to extend the list of data categories. Thus, structuring the classes after the state allows us to plug another data category by inheriting these base abstract classes.

Let's see the complete code for the hierarchy of the cleaned document. All the attributes of a cleaned document will be saved within the metadata of the vector DB. For example, the metadata of a cleaned article document will always contain the content, platform, author ID, author full name, and link of the article.

Another fundamental aspect is the `Config` internal class, which defines the name of the collection within the vector DB, the data category of the entity, and whether to leverage the vector index when creating the collection:

```
class CleanedDocument(VectorBaseDocument, ABC):
    content: str
    platform: str
    author_id: UUID4
    author_full_name: str
class CleanedPostDocument(CleanedDocument):
    image: Optional[str] = None
    class Config:
```

```
        name = "cleaned_posts"
        category = DataCategory.POSTS
        use_vector_index = False
class CleanedArticleDocument(CleanedDocument):
    link: str
    class Config:
        name = "cleaned_articles"
        category = DataCategory.ARTICLES
        use_vector_index = False
class CleanedRepositoryDocument(CleanedDocument):
    name: str
    link: str
    class Config:
        name = "cleaned_repositories"
        category = DataCategory.REPOSITORIES
        use_vector_index = False
```

To conclude this section, let's also take a look at the base abstract class of the chunk and embedded chunk:

```
class Chunk(VectorBaseDocument, ABC):
    content: str
    platform: str
    document_id: UUID4
    author_id: UUID4
    author_full_name: str
    metadata: dict = Field(default_factory=dict)
… # PostChunk, ArticleChunk, RepositoryChunk
class EmbeddedChunk(VectorBaseDocument, ABC):
    content: str
    embedding: list[float] | None
    platform: str
    document_id: UUID4
    author_id: UUID4
    author_full_name: str
    metadata: dict = Field(default_factory=dict)
… # EmbeddedPostChunk, EmbeddedArticleChunk, EmbeddedRepositoryChunk
```

We also defined an enum that aggregates all our data categories in a single structure of constants:

```
class DataCategory(StrEnum):
    POSTS = "posts"
    ARTICLES = "articles"
    REPOSITORIES = "repositories"
```

The last step to fully understand how the domain objects work is to zoom into the `VectorBaseDocument` OVM class.

OVM

The term OVM is inspired by the **object-relational mapping** (**ORM**) pattern we discussed in *Chapter 3*. We called it OVM because we work with embedding and vector DBs instead of structured data and SQL tables. Otherwise, it follows the same principles as an ORM pattern.

Similar to what we did in *Chapter 3*, we will implement our own OVM version. Even if our custom example is simple, it's a powerful example of

how to write modular and extendable classes by leveraging OOP best practices and principles.

The full implementation of the `VectorBaseDocument` class is available on GitHub at https://github.com/PacktPublishing/LLM-Engineering/blob/main/llm_engineering/domain/base/vector.py.

Our OVM base class is called `VectorBaseDocument`. It will support CRUD operations on top of Qdrant. Based on our application's demands, we limited it only to create and read operations, but it can easily be extended to update and delete functions.

Let's take a look at the definition of the `VectorBaseDocument` class:

```
from pydantic import UUID4, BaseModel
from typing import Generic
from llm_engineering.infrastructure.db.qdrant import connection
T = TypeVar("T", bound="VectorBaseDocument")
class VectorBaseDocument(BaseModel, Generic[T], ABC):
    id: UUID4 = Field(default_factory=uuid.uuid4)
    @classmethod
    def from_record(cls: Type[T], point: Record) -> T:
        _id = UUID(point.id, version=4)
        payload = point.payload or {}
        attributes = {
            "id": _id,
            **payload,
        }
        if cls._has_class_attribute("embedding"):
            payload["embedding"] = point.vector or None
        return cls(**attributes)
    def to_point(self: T, **kwargs) -> PointStruct:
        exclude_unset = kwargs.pop("exclude_unset", False)
        by_alias = kwargs.pop("by_alias", True)
        payload = self.dict(exclude_unset=exclude_unset, by_alias=by_alias, **kwar
        _id = str(payload.pop("id"))
        vector = payload.pop("embedding", {})
        if vector and isinstance(vector, np.ndarray):
            vector = vector.tolist()
        return PointStruct(id=_id, vector=vector, payload=payload)
```

- The `VectorBaseDocument` class inherits from Pydantic's `BaseModel` and helps us structure a single record's attributes from the vector DB. Every OVM will be initialized by default with UUID4 as its unique identifier. Using generics—more precisely, by inheriting from `Generic[T]`—the signatures of all the subclasses of the `VectorBaseDocument` class will adapt to that given class. For example, the `from_record()` method of the `Chunk()` class, which inherits `VectorBaseDocument`, will return the Chunk type, which drastically helps the static analyzer and type checkers such as `mypy` (https://mypy.readthedocs.io/en/stable/).

The `from_record()` method adapts a data point from Qdrant's format to our internal structure based on Pydantic. On the other hand, the `to_point()` method takes the attributes of the current instance and adapts

them to Qdrant's `PointStruct()` format. We will leverage these two methods for our create and read operations.

Ultimately, all operations made to Qdrant will be done through the `connection` instance, which is instantiated in the application's infrastructure layer.

The `bulk_insert()` method maps each document to a point. Then, it uses the Qdrant `connection` instance to load all the points to a given collection in Qdrant. If the insertion fails once, it tries to create the collection and do the insertion again. Often, it is good practice to split your logic into two functions. One private function contains the logic, in our case `_bulk_insert()`, and one public function handles all the errors and failure scenarios.

```
class VectorBaseDocument(BaseModel, Generic[T], ABC):
    … # Rest of the class
    @classmethod
    def bulk_insert(cls: Type[T], documents: list["VectorBaseDocument"]) -> bool:
        try:
            cls._bulk_insert(documents)
        except exceptions.UnexpectedResponse:
            logger.info(
                f"Collection '{cls.get_collection_name()}' does not exist. Trying t
            )
            cls.create_collection()
            try:
                cls._bulk_insert(documents)
            except exceptions.UnexpectedResponse:
                logger.error(f"Failed to insert documents in '{cls.get_collection_n
                return False
        return True
    @classmethod
    def _bulk_insert(cls: Type[T], documents: list["VectorBaseDocument"]) -> None:
        points = [doc.to_point() for doc in documents]
        connection.upsert(collection_name=cls.get_collection_name(), points=points)
```

The collection name is inferred from the `Config` class defined in the subclasses inheriting the OVM:

```
class VectorBaseDocument(BaseModel, Generic[T], ABC):
    … # Rest of the class
    @classmethod
    def get_collection_name(cls: Type[T]) -> str:
        if not hasattr(cls, "Config") or not hasattr(cls.Config, "name"):
            raise ImproperlyConfigured(
                "The class should define a Config class with" "the 'name' property
            )
        return cls.Config.name
```

Now, we must define a method that lets us read all the records from the vector DB (without using vector similarity search logic). The `bulk_find()` method enables us to scroll (or list) all the records from a collection. The function below scrolls the Qdrant vector DB, which returns a list of data points, which are ultimately mapped to our internal structure using the `from_record()` method.

The limit parameters control how many items we return at once, and the offset signals the ID of the point from which Qdrant starts returning

records.

```
class VectorBaseDocument(BaseModel, Generic[T], ABC):
    … # Rest of the class
    @classmethod
    def bulk_find(cls: Type[T], limit: int = 10, **kwargs) -> tuple[list[T], UUID
        try:
            documents, next_offset = cls._bulk_find(limit=limit, **kwargs)
        except exceptions.UnexpectedResponse:
            logger.error(f"Failed to search documents in '{cls.get_collection_name
            documents, next_offset = [], None
        return documents, next_offset
    @classmethod
    def _bulk_find(cls: Type[T], limit: int = 10, **kwargs) -> tuple[list[T], UUID
        collection_name = cls.get_collection_name()
        offset = kwargs.pop("offset", None)
        offset = str(offset) if offset else None
        records, next_offset = connection.scroll(
            collection_name=collection_name,
            limit=limit,
            with_payload=kwargs.pop("with_payload", True),
            with_vectors=kwargs.pop("with_vectors", False),
            offset=offset,
            **kwargs,
        )
        documents = [cls.from_record(record) for record in records]
        if next_offset is not None:
            next_offset = UUID(next_offset, version=4)
        return documents, next_offset
```

The last piece of the puzzle is to define a method that performs a vector similarity search on a provided query embedding. Like before, we defined a public `search()` and private `_search()` method. The search is performed by Qdrant when calling the `connection.search()` function.

```
class VectorBaseDocument(BaseModel, Generic[T], ABC):
    … # Rest of the class
    @classmethod
    def search(cls: Type[T], query_vector: list, limit: int = 10, **kwargs) -> list
        try:
            documents = cls._search(query_vector=query_vector, limit=limit, **kwarg
        except exceptions.UnexpectedResponse:
            logger.error(f"Failed to search documents in '{cls.get_collection_name
            documents = []
        return documents
    @classmethod
    def _search(cls: Type[T], query_vector: list, limit: int = 10, **kwargs) -> li
        collection_name = cls.get_collection_name()
        records = connection.search(
            collection_name=collection_name,
            query_vector=query_vector,
            limit=limit,
            with_payload=kwargs.pop("with_payload", True),
            with_vectors=kwargs.pop("with_vectors", False),
            **kwargs,
        )
        documents = [cls.from_record(record) for record in records]
        return documents
```

Now that we understand what our domain entities look like and how the OVM works, let's move on to the dispatchers who clean, chunk, and em-

bed the documents.

The dispatcher layer

A dispatcher inputs a document and applies dedicated handlers based on its data category (article, post, or repository). A handler can either clean, chunk, or embed a document.

Let's start by zooming in on the `CleaningDispatcher`. It mainly implements a `dispatch()` method that inputs a raw document. Based on its data category, it instantiates and calls a handler that applies the cleaning logic specific to that data point:

```
class CleaningDispatcher:
    cleaning_factory = CleaningHandlerFactory()
    @classmethod
    def dispatch(cls, data_model: NoSQLBaseDocument) -> VectorBaseDocument:
        data_category = DataCategory(data_model.get_collection_name())
        handler = cls.cleaning_factory.create_handler(data_category)
        clean_model = handler.clean(data_model)
        logger.info(
            "Data cleaned successfully.",
            data_category=data_category,
            cleaned_content_len=len(clean_model.content),
        )
        return clean_model
```

The key in the dispatcher logic is the `CleaningHandlerFactory()`, which instantiates a different cleaning handler based on the document's data category:

```
class CleaningHandlerFactory:
    @staticmethod
    def create_handler(data_category: DataCategory) -> CleaningDataHandler:
        if data_category == DataCategory.POSTS:
            return PostCleaningHandler()
        elif data_category == DataCategory.ARTICLES:
            return ArticleCleaningHandler()
        elif data_category == DataCategory.REPOSITORIES:
            return RepositoryCleaningHandler()
        else:
            raise ValueError("Unsupported data type")
```

The Dispatcher or Factory classes are nothing fancy, but they offer an intuitive and simple interface for applying various operations to your documents. When manipulating documents, instead of worrying about their data category and polluting your business logic with if-else statements, you have a class dedicated to handling that. You have a single class that cleans any document, which respects the DRY (don't repeat yourself) principles from software engineering. By respecting DRY, you have a single point of failure, and the code can easily be extended. For example, if we add an extra type, we must extend only the Factory class instead of multiple occurrences in the code.

The `ChunkingDispatcher` and `EmbeddingDispatcher` follow the same pattern. They use a `ChunkingHandlerFactory` and, respectively, an `EmbeddingHandlerFactory` that initializes the correct handler

based on the data category of the input document. Afterward, they call the handler and return the result.

The source code of all the dispatchers and factories can be found on GitHub at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/preprocessing/dispatchers.py

The Factory class leverages theabstract factory creational pattern (https://refactoring.guru/design-patterns/abstract-factory), which instantiates a family of classes implementing the same interface. In our case, these handlers implement the `clean()` method regardless of the handler type.

Also, the Handler class family leverages the strategy behavioral pattern (https://refactoring.guru/design-patterns/strategy) used to instantiate when you want to use different variants of an algorithm within an object and be able to switch from one algorithm to another during runtime.

Intuitively, in our dispatcher layer, the combination of the factory and strategy patterns works as follows:

1. Initially, we knew we wanted to clean the data, but as we knew the data category only at runtime, we couldn't decide on what strategy to apply.
2. We can write the whole code around the cleaning code and abstract away the logic under a `Handler()` interface, which will represent our strategy.
3. When we get a data point, we apply the abstract factory pattern and create the correct cleaning handler for its data type.
4. Ultimately, the dispatcher layer uses the handler and executes the right strategy.

By doing so, we:

- Isolate the logic for a given data category.
- Leverage polymorphism to avoid filling up the code with hundreds of `if-else` statements.
- Make the code modular and extendable. When a new data category arrives, we must implement a new handler and modify the Factory class without touching any other part of the code.

Until now, we have just modeled our entities and how the data flows in our application. We haven't written a single piece of cleaning, chunking, or embedding code. That is one big difference between a quick demo and a production-ready application. In a demo, you don't care about software engineering best practices and structuring your code to make it future-proof. However, writing clean, modular, and scalable code is critical for its longevity when building a real-world application.

The last component of the RAG feature pipeline is the implementation of the cleaning, chunking, and embedding handlers.

The handlers

The handler has a one-on-one structure with our domain, meaning that every entity has its own handler, as shown in Figure 8.17. In total, we will have nine Handler classes that follow the next base interfaces:

- `class CleaningDataHandler()`
- `class ChunkingDataHandler()`
- `class EmbeddingDataHandler()`

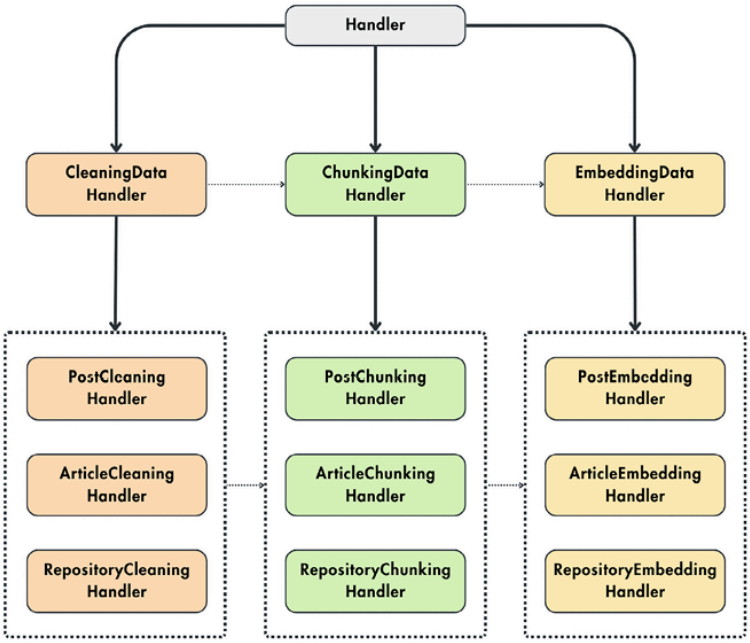

Figure 4.17: Handler class hierarchy and their interaction

The code for all the handlers is available on GitHub at https://github.com/PacktPublishing/LLM-Engineering/tree/main/llm_engineering/application/preprocessing.

Let's examine each handler family and see how it is implemented.

The cleaning handlers

The `CleaningDataHandler()` strategy interface looks as follows:

```
… # Other imports.
from typing import Generic, TypeVar
DocumentT = TypeVar("DocumentT", bound=Document)
CleanedDocumentT = TypeVar("CleanedDocumentT", bound=CleanedDocument)
class CleaningDataHandler(ABC, Generic[DocumentT, CleanedDocumentT]):
    @abstractmethod
    def clean(self, data_model: DocumentT) -> CleanedDocumentT:
        pass
```

Now, for every post, article and repository, we have to implement a different handler, as follows:

```
class PostCleaningHandler(CleaningDataHandler):
    def clean(self, data_model: PostDocument) -> CleanedPostDocument:
```

```
        return CleanedPostDocument(
            id=data_model.id,
            content=clean_text(" #### ".join(data_model.content.values())),
            … # Copy the rest of the parameters from the data_model object.
        )
class ArticleCleaningHandler(CleaningDataHandler):
    def clean(self, data_model: ArticleDocument) -> CleanedArticleDocument:
        valid_content = [content for content in data_model.content.values() if cont
        return CleanedArticleDocument(
            id=data_model.id,
            content=clean_text(" #### ".join(valid_content)),
            platform=data_model.platform,
            link=data_model.link,
            author_id=data_model.author_id,
            author_full_name=data_model.author_full_name,
        )
class RepositoryCleaningHandler(CleaningDataHandler):
    def clean(self, data_model: RepositoryDocument) -> CleanedRepositoryDocument:
        return CleanedRepositoryDocument(
            id=data_model.id,
            content=clean_text(" #### ".join(data_model.content.values())),
            … # Copy the rest of the parameters from the data_model object.
        )
```

The handlers input a raw document domain entity, clean the content, and return a cleaned document. All the handlers use the `clean_text()` function to clean the text. Out of simplicity, we used the same cleaning technique for all the data categories. Still, in a real-world setup, we would have to further optimize and create a different cleaning function for each data category. The strategy pattern makes this a breeze, as we swap the cleaning function in the handlers, and that's it.

The cleaning steps applied in the `clean_text()` function are the same ones discussed in *Chapter 5* in the *Creating an instruction dataset* section. We don't want to repeat ourselves. Thus, for a refresher, check out that chapter. At this point, we mostly care about automating and integrating the whole logic into the RAG feature pipeline. Thus, after operationalizing the ML system, all the cleaned data used for fine-tuning will be accessed from the logical feature store, making it the single source of truth for accessing data.

The chunking handlers

First, let's examine the `ChunkingDataHandler()` strategy handler. We exposed the `metadata` dictionary as a property to aggregate all the necessary properties required for chunking in a single structure. By structuring it like this, we can easily log everything to ZenML to track and debug our chunking logic. The handler takes cleaned documents as input and returns chunk entities. All the handlers can be found on GitHub at https://github.com/PacktPublishing/LLM-Engineering/tree/main/llm_engineering/application/preprocessing.

```
… # Other imports.
from typing import Generic, TypeVar
CleanedDocumentT = TypeVar("CleanedDocumentT", bound=CleanedDocument)
ChunkT = TypeVar("ChunkT", bound=Chunk)
 class ChunkingDataHandler(ABC, Generic[CleanedDocumentT, ChunkT]):
    @property
    def metadata(self) -> dict:
```

```
        return {
            "chunk_size": 500,
            "chunk_overlap": 50,
        }
    @abstractmethod
    def chunk(self, data_model: CleanedDocumentT) -> list[ChunkT]:
        pass
```

Let's understand how the `ArticleChunkingHandler()` class is implemented. The first step is to override the metadata property and customize the type of properties the chunking logic requires. For example, when working with articles, we are interested in the chunk's minimum and maximum length.

The handler's `chunk()` method inputs cleaned article documents and returns a list of article chunk entities. It uses the `chunk_text()` function to split the cleaned content into chunks. The chunking function is customized based on the `min_length` and `max_length` metadata fields. The chunk_id is computed as the MD5 hash of the chunk's content. Thus, if the two chunks have precisely the same content, they will have the same ID, and we can easily deduplicate them. Lastly, we create a list of chunk entities and return them.

```
class ArticleChunkingHandler(ChunkingDataHandler):
    @property
    def metadata(self) -> dict:
        return {
            "min_length": 1000,
            "max_length": 1000,
        }
    def chunk(self, data_model: CleanedArticleDocument) -> list[ArticleChunk]:
        data_models_list = []
        cleaned_content = data_model.content
        chunks = chunk_article(
            cleaned_content, min_length=self.metadata["min_length"], max_length=se
        )
        for chunk in chunks:
            chunk_id = hashlib.md5(chunk.encode()).hexdigest()
            model = ArticleChunk(
                id=UUID(chunk_id, version=4),
                content=chunk,
                platform=data_model.platform,
                link=data_model.link,
                document_id=data_model.id,
                author_id=data_model.author_id,
                author_full_name=data_model.author_full_name,
                metadata=self.metadata,
            )
            data_models_list.append(model)
        return data_models_list
```

The last step is to dig into the `chunk_article()` function, which mainly does two things:

- It uses a regex to find all the sentences within the given text by looking for periods, question marks, or exclamation points followed by a space. However, it avoids splitting into cases where the punctuation is part of an abbreviation or initialism (like "`e.g.`" or "`Dr.`")

- It groups sentences into a single chunk until the `max_length` limit is reached. When the maximum size is reached, and the chunk size is bigger than the minimum allowed value, it is added to the final list the function returns.

```
def chunk_article(text: str, min_length: int, max_length: int) -> list[str]:
    sentences = re.split(r"(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?|\!)\s", text)
    extracts = []
    current_chunk = ""
    for sentence in sentences:
        sentence = sentence.strip()
        if not sentence:
            continue
        if len(current_chunk) + len(sentence) <= max_length:
            current_chunk += sentence + " "
        else:
            if len(current_chunk) >= min_length:
                extracts.append(current_chunk.strip())
            current_chunk = sentence + " "
    if len(current_chunk) >= min_length:
        extracts.append(current_chunk.strip())
    return extracts
```

The `PostChunkingHandler` and `RepositoryChunkingHandler`, available on GitHub at `llm_engineering/application/preprocessing/chunking_data_handlers.py`, have a similar structure to the ArticleChunkingHandler. However, they use a more generic chunking function called `chunk_text()`, worth looking into. The `chunk_text()` function is a two-step process that has the following logic:

1. It uses a `RecursiveCharacterTextSplitter()` from LangChain to split the text based on a given separator or chunk size. Using the separator, we first try to find paragraphs in the given text, but if there are no paragraphs or they are too long, we cut it at a given chunk size.
2. Notice that we want to ensure that the chunk doesn't exceed the maximum input length of the embedding model. Thus, we pass all the chunks created above into a `SenteceTransformersTokenTextSplitter()`, which considers the maximum input length of the model. At this point, we also apply the `chunk_overlap` logic, as we want to do it only after we validate that the chunk is small enough.

```
… # Other imports.
from langchain.text_splitter import RecursiveCharacterTextSplitter, SentenceTran
from llm_engineering.application.networks import EmbeddingModelSingleton
def chunk_text(text: str, chunk_size: int = 500, chunk_overlap: int = 50) -> lis
    character_splitter = RecursiveCharacterTextSplitter(separators=["\n\n"], chu
    text_split_by_characters = character_splitter.split_text(text)
    token_splitter = SentenceTransformersTokenTextSplitter(
        chunk_overlap=chunk_overlap,
        tokens_per_chunk=embedding_model.max_input_length,
        model_name=embedding_model.model_id,
    )
    chunks_by_tokens = []
    for section in text_split_by_characters:
        chunks_by_tokens.extend(token_splitter.split_text(section))
    return chunks_by_tokens
```

To conclude, the function above returns a list of chunks that respect both the provided chunk parameters and the embedding model's max input length.

The embedding handlers

The embedding handlers differ slightly from the others as the `EmbeddingDataHandler()` interface contains most of the logic. We took this approach because, when calling the embedding model, we want to batch as many samples as possible to optimize the inference process. When running the model on a GPU, the batched samples are processed independently and in parallel. Thus, by batching the chunks, we can optimize the inference process by 10x or more, depending on the batch size and hardware we use.

We implemented an `embed()` method, in case you want to run the inference on a single data point, and an `embed_batch()` method. The `embed_batch()` method takes chunked documents as input, gathers their content into a list, passes them to the embedding model, and maps the results to an embedded chunk domain entity. The mapping is done through the `map_model()` abstract method, which has to be customized for every data category.

```
… # Other imports.
from typing import Generic, TypeVar, cast
from llm_engineering.application.networks import EmbeddingModelSingleton
ChunkT = TypeVar("ChunkT", bound=Chunk)
EmbeddedChunkT = TypeVar("EmbeddedChunkT", bound=EmbeddedChunk)
embedding_model = EmbeddingModelSingleton()
class EmbeddingDataHandler(ABC, Generic[ChunkT, EmbeddedChunkT]):
    """
    Abstract class for all embedding data handlers.
    All data transformations logic for the embedding step is done here
    """
    def embed(self, data_model: ChunkT) -> EmbeddedChunkT:
        return self.embed_batch([data_model])[0]
    def embed_batch(self, data_model: list[ChunkT]) -> list[EmbeddedChunkT]:
        embedding_model_input = [data_model.content for data_model in data_model]
        embeddings = embedding_model(embedding_model_input, to_list=True)
        embedded_chunk = [
            self.map_model(data_model, cast(list[float], embedding))
            for data_model, embedding in zip(data_model, embeddings, strict=False)
        ]
        return embedded_chunk
    @abstractmethod
    def map_model(self, data_model: ChunkT, embedding: list[float]) -> EmbeddedChu
        pass
```

Let's look only at the implementation of the `ArticleEmbeddingHandler()`, as the other handlers are highly similar. As you can see, we only have to implement the `map_model()` method, which takes a chunk of input and computes the embeddings in batch mode. Its scope is to map this information to an `EmbeddedArticleChunk` Pydantic entity.

```
class ArticleEmbeddingHandler(EmbeddingDataHandler):
    def map_model(self, data_model: ArticleChunk, embedding: list[float]) -> Embed
        return EmbeddedArticleChunk(
            id=data_model.id,
```

```
            content=data_model.content,
            embedding=embedding,
            platform=data_model.platform,
            link=data_model.link,
            document_id=data_model.document_id,
            author_id=data_model.author_id,
            author_full_name=data_model.author_full_name,
            metadata={
                "embedding_model_id": embedding_model.model_id,
                "embedding_size": embedding_model.embedding_size,
                "max_input_length": embedding_model.max_input_length,
            },
        )
```

The last step is to understand how the `EmbeddingModelSingleton()` works. It is a wrapper over the `SentenceTransformer()` class from Sentence Transformers that initializes the embedding model. Writing a wrapper over external packages is often good practice. Thus, when you want to change the third-party tool, you have to modify only the internal logic of the wrapper instead of the whole code base.

The `SentenceTransformer()` class is initialized with the `model_id` defined in the `Settings` class, allowing us to quickly test multiple embedding models just by changing the configuration file and not the code. That is why I am not insisting at all on what embedding model to use. This differs constantly based on your use case, data, hardware, and latency. But by writing a generic class, which can quickly be configured, you can experiment with multiple embedding models until you find the best one for you.

```
from sentence_transformers.SentenceTransformer import SentenceTransformer
from llm_engineering.settings import settings
from .base import SingletonMeta
class EmbeddingModelSingleton(metaclass=SingletonMeta):
    def __init__(
        self,
        model_id: str = settings.TEXT_EMBEDDING_MODEL_ID,
        device: str = settings.RAG_MODEL_DEVICE,
        cache_dir: Optional[Path] = None,
    ) -> None:
        self._model_id = model_id
        self._device = device
        self._model = SentenceTransformer(
            self._model_id,
            device=self._device,
            cache_folder=str(cache_dir) if cache_dir else None,
        )
        self._model.eval()
    @property
    def model_id(self) -> str:
        return self._model_id
    @cached_property
    def embedding_size(self) -> int:
        dummy_embedding = self._model.encode("")
        return dummy_embedding.shape[0]
    @property
    def max_input_length(self) -> int:
        return self._model.max_seq_length
    @property
    def tokenizer(self) -> AutoTokenizer:
        return self._model.tokenizer
```

```
    def __call__(
        self, input_text: str | list[str], to_list: bool = True
    ) -> NDArray[np.float32] | list[float] | list[list[float]]:
        try:
            embeddings = self._model.encode(input_text)
        except Exception:
            logger.error(f"Error generating embeddings for {self._model_id=} and {:
            return [] if to_list else np.array([])
        if to_list:
            embeddings = embeddings.tolist()
        return embeddings
```

The embedding model class implements the singleton pattern (https://refactoring.guru/design-patterns/singleton), a creational design pattern that ensures a class has only one instance while providing a global access point to this instance. The `EmbeddingModelSingleton()` class inherits from the `SingletonMeta` class, which ensures that whenever an `EmbeddingModelSingleton()` is instantiated, it returns the same instance. This works well with ML models, as you load them once in memory through the singleton pattern, and afterward, you can use them anywhere in the code base. Otherwise, you risk loading the model in memory every time you use it or loading it multiple times, resulting in memory issues. Also, this makes it very convenient to access properties such as `embedding_size`, where you have to make a dummy forward pass into the embedding model to find the size of its output. As a singleton, you do this forward pass only once, and then you have it accessible all the time during the program's execution.

Summary

This chapter began with a soft introduction to RAG and why and when you should use it. We also understood how embeddings and vector DBs work, representing the cornerstone of any RAG system. Then, we looked into advanced RAG and why we need it in the first place. We built a strong understanding of what parts of the RAG can be optimized and proposed some popular advanced RAG techniques for working with textual data. Next, we applied everything we learned about RAG to designing the architecture of LLM Twin's RAG feature pipeline. We also understood the difference between a batch and streaming pipeline and presented a short introduction to the CDC pattern, which helps sync two DBs.

Ultimately, we went step-by-step into the implementation of the LLM Twin's RAG feature pipeline, where we saw how to integrate ZenML as an orchestrator, how to design the domain entities of the application, and how to implement an OVM module. Also, we understood how to apply some software engineering best practices, such as the abstract factory and strategy software patterns, to implement a modular and extendable layer that applies different cleaning, chunking, and embedding techniques based on the data category of each document.

This chapter focused only on implementing the ingestion pipeline, which is just one component of a standard RAG application. In *Chapter 9*, we will conclude the RAG system by implementing the retrieval and generation components and integrating them into the inference pipeline. But first, in the next chapter, we will explore how to generate a custom dataset using the data we collected and fine-tune an LLM with it.

References

- Kenton, J.D.M.W.C. and Toutanova, L.K., 2019, June. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT* (Vol. 1, p. 2).
- Liu, Y., 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Mikolov, T., 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. *GloVe: Global Vectors for Word Representation*. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing* (*EMNLP*), pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
- Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J. and Krueger, G., 2021, July. Learning transferable visual models from natural language supervision. In *International conference on machine learning* (pp. 8748-8763). PMLR.
- *What is Change Data Capture (CDC)? | Confluent*. (n.d.). Confluent. https://www.confluent.io/en-gb/learn/change-data-capture/
- Refactoring.Guru. (2024, January 1). *Singleton*. https://refactoring.guru/design-patterns/singleton
- Refactoring.Guru. (2024b, January 1). *Strategy*. https://refactoring.guru/design-patterns/strategy
- Refactoring.Guru. (2024a, January 1). *Abstract Factory*. https://refactoring.guru/design-patterns/abstract-factory
- Schwaber-Cohen, R. (n.d.). *What is a Vector Database & How Does it Work? Use Cases + Examples*. Pinecone. https://www.pinecone.io/learn/vector-database/
- Monigatti, L. (2024, February 19). *Advanced Retrieval-Augmented Generation: From Theory to LlaMaIndex Implementation*. *Medium*. https://towardsdatascience.com/advanced-retrieval-augmented-generation-from-theory-to-llamaindex-implementation-4de1464a9930
- Monigatti, L. (2023, December 6). A guide on 12 tuning Strategies for Production-Ready RAG applications. *Medium*. https://towardsdatascience.com/a-guide-on-12-tuning-strategies-for-production-ready-rag-applications-7ca646833439
- Monigatti, L. (2024b, February 19). *Advanced Retrieval-Augmented Generation: From Theory to LlaMaIndex Implementation*. *Medium*. https://towardsdatascience.com/advanced-retrieval-augmented-generation-from-theory-to-llamaindex-implementation-4de1464a9930
- Maameri, S. (2024, May 10). Routing in RAG-Driven applications - towards data science. *Medium*. https://towardsdatascience.com/routing-in-rag-driven-applications-a685460a7220

Join our book's Discord space

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

Answers collapsed